# 國 立 中 正 大 學

資訊工程研究所碩士論文

# 使用 16 奈米 CMOS 技術及提前退出機制實現之低功耗分支卷積神經網路硬體加速器用於無人機災害檢測

# Low-Power Branch CNN Hardware Accelerator with Early Exit for UAV Disaster Detection Using 16nm CMOS Technology

研 究 生 ： 晁文勤

指導教授 ： 鍾菁哲 博士

中華民國 一一三 年 八 月

# 國立中正大學碩士學位論文考試審定書

資訊工程學系

研究生 晁文勤 所提之論文

使用16奈米CMOS技術及提前退出機制實現之低功耗分支卷積神經網路硬體加速器用於無人機災害檢測

經本委員會審查，符合 碩 士學位論文標準。

學位考試委員會
召集人 李順裕 簽章

委員 李順裕

黃宗勳 岱 崙

陳雲岫

指導教授 陳雲岫 簽章

中華民國 113 年 6 月 27 日

# 摘要

在現代災害管理中，無人機和機器學習的結合正在改變我們應對災害的方式。無人機能夠快速部署到災區，提供高解析度的即時影像和數據。而機器學習則能夠從無人機收集的圖像中自動識別災害類型、受災區域的範圍以及受影響的基礎設施，並且迅速分析這些數據以制定有效的救援策略，這極大地縮短了人為分析的時間。這些即時訊息對於救援隊伍在第一時間做出決策至關重要，如搜索倖存者、分配資源和規劃撤離路線。

本論文根據災害事故的空拍圖像來進行檢測，對災害的發生進行即時的應變與管理。所提出的分支式卷積神經網路利用分支學習來增強特徵學習，使模型能以少量參數進行訓練和預測。此外，為了減少記憶體大小以及計算量，本論文使用 DoReFa-Net 量化權重以及定點化量化模型參數，並且具有提前退出機制，從而使卷積神經網路能夠以低成本與低功率的方式實現。

本論文所提出的分支式卷積神經網路硬體加速器在 TSMC 16nm CMOS 技術下實現，並使用電源閘控管技術管理記憶體的電源開關，以更進一步達成低功耗。經由自動布局繞線實現後，工作頻率可以達到 500 MHz，功耗為 37.56 mW，同時電路預測災害事故的準確率為 88.18%。

**關鍵字**：無人機、災害偵測、分支式卷積神經網路、提前退出機制、DoReFa-Net、低功耗晶片

# Abstract

In modern disaster management, the combination of unmanned aerial vehicles (UAVs) and machine learning is transforming how we respond to disasters. UAVs can be quickly deployed to disaster areas, providing high-resolution real-time images and data. Machine learning can then automatically identify disaster types, affected areas, and impacted infrastructure from the images collected by UAVs. This rapid analysis helps create effective rescue strategies, significantly reducing the time required for human analysis. These real-time insights are crucial for rescue teams to make immediate decisions, such as searching for survivors, allocating resources, and planning evacuation routes.

This thesis focuses on detecting disasters using aerial images. The proposed Branch Convolutional Neural Network (B-CNN) enhances feature learning through branch training, allowing the model to train and predict with fewer parameters. To reduce memory usage and computational load, DoReFa-Net quantizes weights, and fixed-point quantizes model parameters, with an early exit mechanism enabling low-cost, low-power implementation of the CNN.

The proposed B-CNN hardware accelerator is implemented using TSMC 16nm CMOS technology and employs power gating to manage memory power switches, achieving low power consumption. After automatic layout and routing, the hardware operates at a frequency of 500 MHz with a power consumption of 37.56 mW. The circuit's accuracy in predicting disaster incidents is 88.18%.

**Keywords:** Unmanned Aerial Vehicles (UAVs), disaster detection, Branch Convolutional Neural Network (B-CNN), early exit mechanism, DoReFa-Net, low-power chip

# Content

# List of Figures

# List of Tables

# Chapter 1 Introduction

## 1.1 Introduction to Unmanned Aerial Vehicles and Disaster Detection

In recent years, there has been significant interest in unmanned aerial vehicles (UAVs) as a remote sensing platform for various purposes. These include tasks such as agricultural surveillance [1], search and rescue missions [2], and traffic management [3], as shown in Figure 1.1. However, traditional cloud computing requires data transfer between the terminal device and the cloud, leading to challenges like data security concerns, privacy preservation, and network delays. UAVs tackle these challenges by autonomously performing computations and processing, enabling faster and more reliable detection and recognition.

Figure 1.1 The sample UAV images of different traffic density [3].

Moreover, UAVs are extensively employed in disaster detection. Firstly, UAVs can provide efficient detection, covering large areas in a short time, enabling quick grasp of disaster situations. This enables us to swiftly obtain a comprehensive overview of the disaster situation, facilitating prompt response measures.

Furthermore, UAVs are capable of navigating various complex geographical environments and terrains, including mountainous regions, water bodies, and urban areas, offering broad applications in disaster monitoring. It is worth noting that UAVs are equipped with high-resolution cameras, providing excellent image capture capabilities to capture detailed imagery. This enhances the precision of disaster assessment. For instance, UAV cameras can clearly capture submerged buildings during floods [4], the formation of landslides after slope collapses [5], and burning areas in wildfires [6], as shown in Figure 1.2. Lastly, UAVs can promptly collect relevant information about the disaster and transmit it to command centers to minimize the losses caused by disasters.

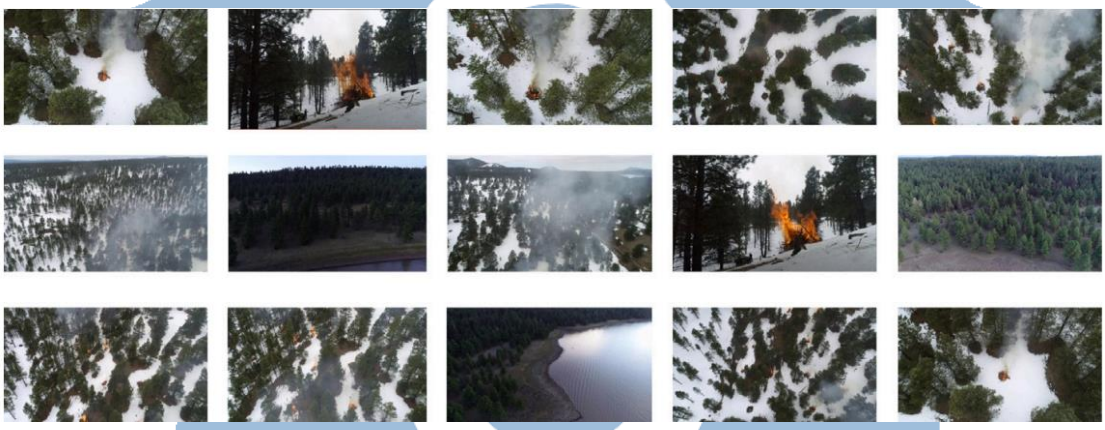


Figure 1.2 The aerial image of wildfire [6].

However, the endurance of UAVs affects their ability for continuous monitoring, coverage range, and detailed observation capability, posing challenges to effectively respond to disasters and reduce losses. For example, a DJI long-range drone [7] has a flight time of 28 minutes with a battery capacity of 4280mAh. If it carries a 5V, 5W Jetson Nano [8], the flight time drops to just 3.7 minutes, reducing by about 87%, and this doesn't even consider the impact of weight. However, if it carries a 40mW Application-Specific Integrated Circuit (ASIC), the flight time can still be maintained at 21 minutes, only reducing by about 25%. Therefore, developing a low-power chip

for drones to extend their flight time is essential for effectively carrying out disaster detection tasks.

Additionally, the aforementioned applications are limited to the detection of individual events, resulting in UAVs being unable to effectively adapt to different environments. This limitation hinders the development of UAVs from disaster prevention and management. Therefore, a specialized database named AIDER [9] (Aerial Image Dataset for Emergency Response applications) has been established, which will be discussed in detail in Section 1.3.

## 1.2 Introduction to Convolution Neural Network

Recently, Convolutional Neural Networks (CNNs) have emerged as a revolutionary method. Modeled after human visual processing, CNNs have exhibited remarkable accomplishments in tasks like image recognition and object detection, and various other visual perception activities. Through features like local perception and weight sharing, CNNs effectively learn from input images and extract their features.

A standard convolutional neural network typically includes an input layer, convolutional layer, activation layer, pooling layer, fully connected layer, and an output layer.

The initial layer of a CNN is structured to manage multi-dimensional data, often presented as images. Each image is represented as a 3D array, with dimensions denoting width, height, and channels. These channels typically convey color details, such as color images with three channels for red, green, and blue (RGB), or grayscale images with one channel, as shown in Figure 1.3.

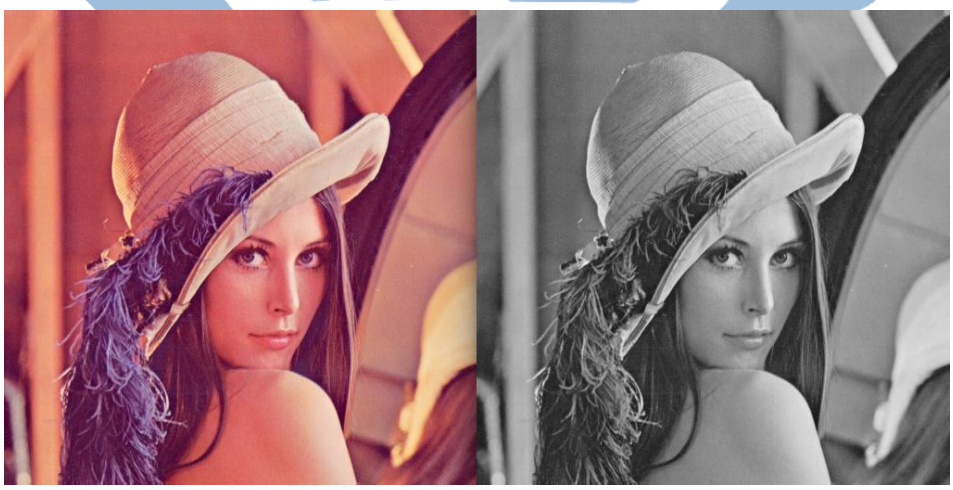


Figure 1.3 The example of color image and grayscale image.

Before performing convolution operations, model training requires resizing images to standardized dimensions to ensure consistent network structure and enhance training efficiency. Common methods for interpolation include Nearest Neighbor,

Bilinear, Area, and Bicubic interpolation. Nearest Neighbor interpolation is the simplest, determining new pixel values based on the nearest existing pixel, which may cause a blocky effect. Bilinear interpolation computes new pixel values by averaging the four closest pixels, resulting in a smoother image. Area interpolation, ideal for reducing image size, averages pixel values across a specified area, thereby preserving more image details. Bicubic interpolation, which considers the 16 nearest pixels, yields the smoothest outcomes but demands more computational resources.

The fundamental process in the convolutional layer is the convolution operation. This process entails moving small filters, also known as kernels, across the input data. These filters are trained to identify patterns like edges, textures, and more intricate features by conducting multiplication and addition with the input data. After the operation of convolution, a feature map is generated, as depicted in Figure 1.4. Each filter within the convolutional layer generates a unique feature map, emphasizing a particular pattern learned during training. Together, these feature maps portray the acquired features of the input data, aiding the network in recognizing complex hierarchical representations.

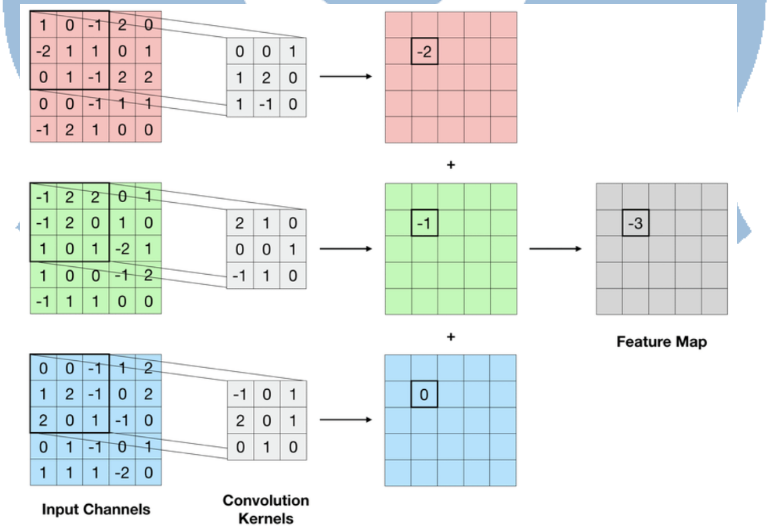



Figure 1.4 The example of covolution operation [10].

After the operation of convolution, the activation layer is applied to add non-

linearity into the model. Without non-linearity, the neural network would operate as a linear model, severely limiting its ability to learn complex patterns and representations. The activation function, utilized to the output of every neuron in the network, serves as a mathematical operation that determines the neuron's output. Common activation functions include ReLU, Sigmoid, and Tanh, as defined in Equations 1.1, 1.2, and 1.3, respectively.

$$ReLU(x) = max(0, x) \tag{1.1}$$

$$Sigmoid(x) = \frac{1}{1+e^{-x}} \tag{1.2}$$

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}} \tag{1.3}$$

The pooling layer serves as a means to decrease the spatial dimensions of the input data, effectively downsampling and preserving crucial information. This reduction in size aids in lowering computational complexity and making the network less reliant on precise spatial feature locations, thereby enhancing translational invariance. Max and average pooling are among the commonly used operations, as depicted in Figure 1.5. Max pooling entails selecting the maximum value from neighboring pixels, retaining prominent features, while average pooling calculates the average value of nearby pixels, resulting in a smoother representation.

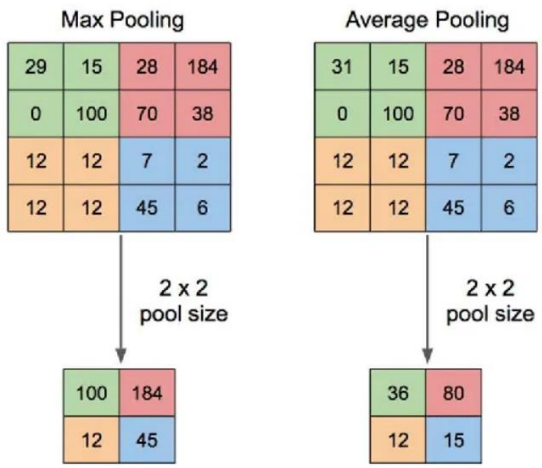



Figure 1.5 The example of max pooling and average pooling [11].

In a CNN structure, the fully connected layer typically follows a sequence of convolution and pooling layers. These initial layers are crafted to autonomously grasp hierarchical features from the input data, including edges, textures, and intricate patterns. Prior to reaching the fully connected layer, the feature maps produced by the convolutional and pooling layers are flattened into a one-dimensional vector. This vector is then subjected to feature fusion through weighted summation. The last layer of the network, referred to as the output layer, is tasked with generating the model's predictions or classifications.

In addition to the mentioned layers, extra techniques can be integrated into neural network models to enhance their performance. For instance, [12] propose a method called Batch Normalization, it can normalizes the input of each layer in a network by adjusting the mean and standard deviation, and it introduces learnable parameters for scaling and shifting. This normalization process helps address internal covariate shift, accelerates training, and improves the stability of deep neural networks.

The batch normalization process is outlined in Equations 1.4 to 1.7, where x denotes the inputs within a mini-batch. The process begins by calculating the mean and variance of the mini-batch, as shown in Equations 1.4 and 1.5. Then, Equation 1.6 demonstrates the normalization of inputs using the previously computed mean and variance. Finally, Equation 1.7 illustrates how the model's feature distribution learning is supported by two training parameters, γ and β.

$$\mu_B \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \tag{1.4}$$

$$\alpha_B^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2 \tag{1.5}$$

$$\hat{x_i} \leftarrow \frac{x_i - \mu_B}{\sqrt{\alpha_B^2 + \epsilon}} \tag{1.6}$$

$$y_i \leftarrow \gamma \hat{x_i} + \beta \equiv BN_{\gamma,\beta}(x_i) \tag{1.7}$$

Global Average Pooling (GAP) [13] is a technique used in Convolutional Neural Networks (CNNs) for spatial dimension reduction and feature summarization. It is often applied as a replacement for or in conjunction with traditional fully connected layers in the final stages of a CNN architecture. Global Average Pooling has gained popularity for its ability to reduce overfitting, improve model interpretability, and reduce the model parameters.

In 1998, Y. Lecun and his team propose a one of most well-known CNN structures for handwritten digit recognition, LeNet-5 [14]. This architecture incorporates two convolutional layers, pooling, fully connected, and Gaussian connection layers. While LeNet-5 may seem relatively simple compared to contemporary CNN architectures, its significance lies in being one of the first successful applications of deep learning to image recognition tasks, paving the path for the evolution of more advanced models in the years to come. The architecture is depicted in Figure 1.6.

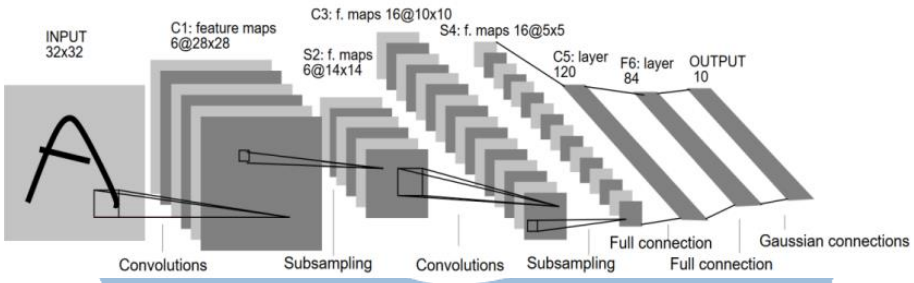



Figure 1.6 The architecture of LeNet-5 CNN used for digit recognition [14].

AlexNet [15], introduced by Alex Krizhevsky and his team, emerged as the winner of the 2012 ImageNet competition. The incorporation of ReLU and Dropout techniques in AlexNet played a significant role in achieving an exceptionally high accuracy rate, consequently popularizing the adoption of ReLU. The architectural design is demonstrated in Figure 1.7.

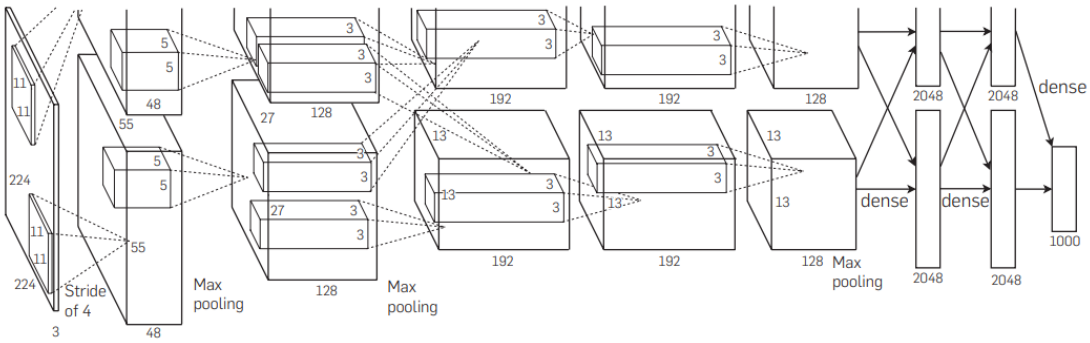



Figure 1.7 The architecture of AlexNet [15].

VGGNet [16], developed by K. Simonyan in 2014, surpasses AlexNet in network depth, opting for a deeper architecture. It employs a 3×3 convolution kernel, replacing the 5×5 convolution kernel used in AlexNet. VGGNet encompasses various structures, such as VGG11, VGG13, VGG16, and VGG19, depending on the number of network layers. The term "VGGNet" commonly refers to the VGG16.

GoogLeNet [17], proposed by C. Szegedy and his team, outperforms VGGNet in the identification of images from the ImageNet dataset. In comparison to AlexNet and VGG, GoogLeNet achieves enhanced breadth. This is accomplished by concatenating convolutions with kernels of different sizes and max-pooling layers into a single output vector, forming what is known as the inception module, as depicted in Figure 1.8. The entirety of GoogLeNet comprises numerous inception modules. Typically, a significant portion of the parameters in a CNN originates from the fully connected (FC) layer. To reduce computational overhead and model parameters, GoogLeNet replaces the FC layer with the global average pooling (GAP) layer.

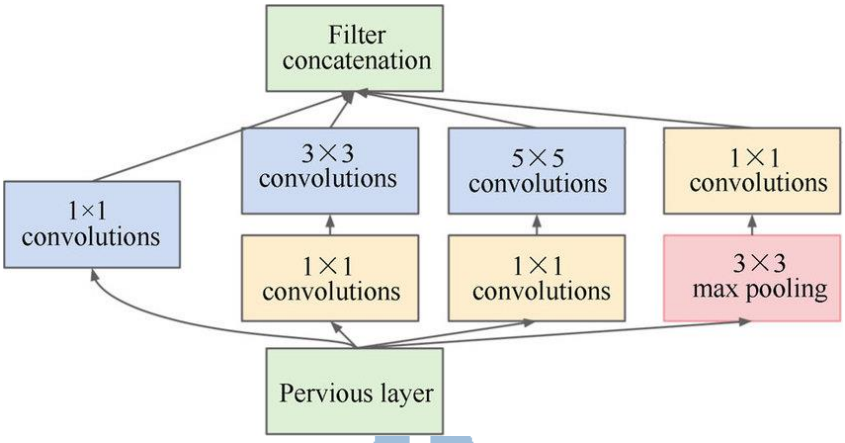



Figure 1.8 Inception module [17].

To tackle the difficulties encountered in training exceptionally deep neural networks, ResNet [18] was developed by K. He and his team. ResNet is known for its innovative use of residual blocks, which enables the training of extremely deep neural networks while addressing the problem of vanishing gradients, as illustrated in Figure 1.9.

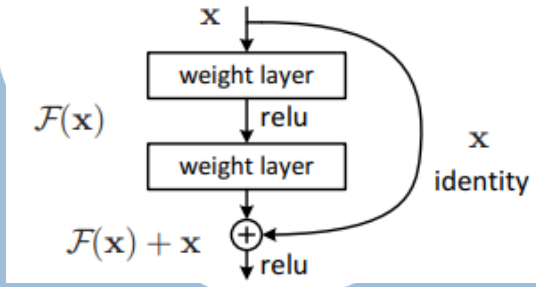


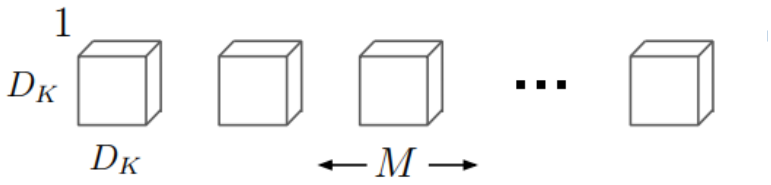

Figure 1.9 Residual block [18].

Nonetheless, VGG, GoogleNet, and ResNet all share common drawbacks primarily concerning their computational demands, model intricacy, and extensive parameter counts. Because of these attributes, these architectures are not ideal for edge computing devices with restricted computational capabilities and memory capacities. These obstacles have spurred the creation of lightweight convolutional neural networks (CNNs) aimed at overcoming these challenges.

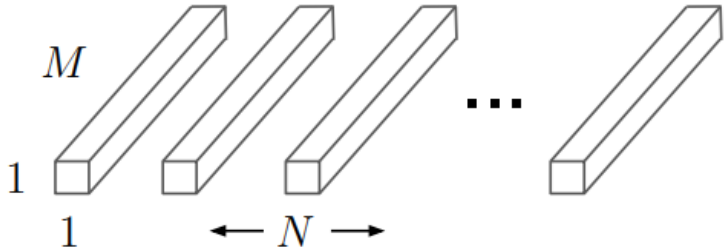
A lightweight convolutional neural network (CNN) is crafted to be

computationally efficient, occupy less memory, and be deployable on devices with limited resources like edge devices, smartphones, and IoT devices. The primary purpose of these lightweight CNNs is to explore the tradeoffs between model complexity and accuracy while keeping computational expenses and memory needs to a minimum. Some popular lightweight CNN architectures include MobileNet [19], ShuffleNet [20], and SqueezeNet [21].

MobileNet, a convolutional neural network created by Google, addresses the issue of parameter count and computations by employing depthwise separable convolutions. This method divides the typical convolution process into depthwise and pointwise convolutions, as depicted in Figure 1.10. The depthwise convolution conducts convolutions separately on each channel, thus lessening the computational burden. On the other hand, the pointwise convolution merges features from various channels using a 1×1 convolutional kernel. Through these techniques, MobileNet efficiently decreases the parameter count and computations while preserving adequate representation capacity.




(a) Depthwise convolution




(b) Pointwise convolution.

Figure 1.10 Depthwise separable convolution [19].

ShuffleNet, a convolutional neural network proposed by the Face++ team, is designed specifically for mobile and edge devices with constrained computing capacity. The key feature of ShuffleNet is the introduction of a novel operation known as channel shuffling, which improves the network's ability to exchange information. This operation is shown in Figure 1.11.

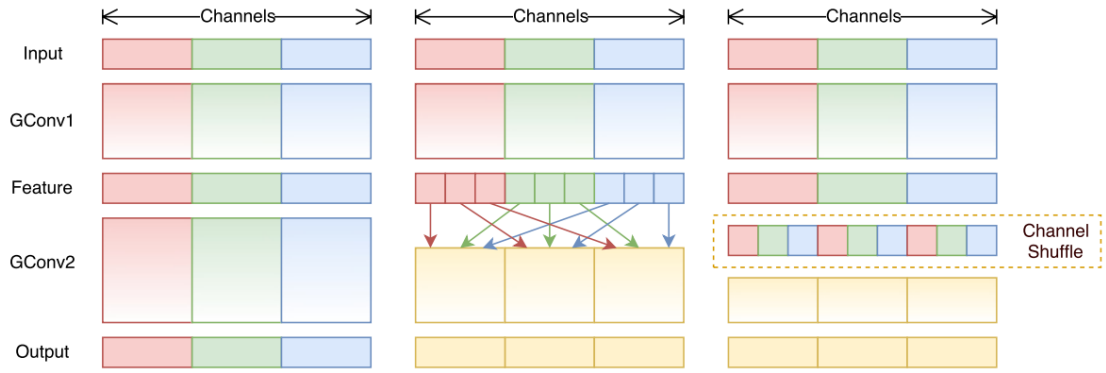



Figure 1.11 Channel shuffling [20].

SqueezeNet, developed by DeepScale, UC Berkeley, and Stanford University, is a lightweight CNN model developed to significantly decrease the model parameters while achieve effectiveness akin to that of AlexNet. Its key innovation lies in the Fire Module, a unique convolutional structure comprising two parts: squeeze and expand. The squeeze part utilizes a 1×1 convolutional kernel to reduce channels in feature maps, thus lowering computational complexity. Conversely, the expand part employs both 1×1 and 3×3 convolutional kernels to increase the number of channels, ensuring adequate representation capacity. This structure is illustrated in Figure 1.12. The integration of Fire Modules effectively reduces parameters while maintaining notable performance.

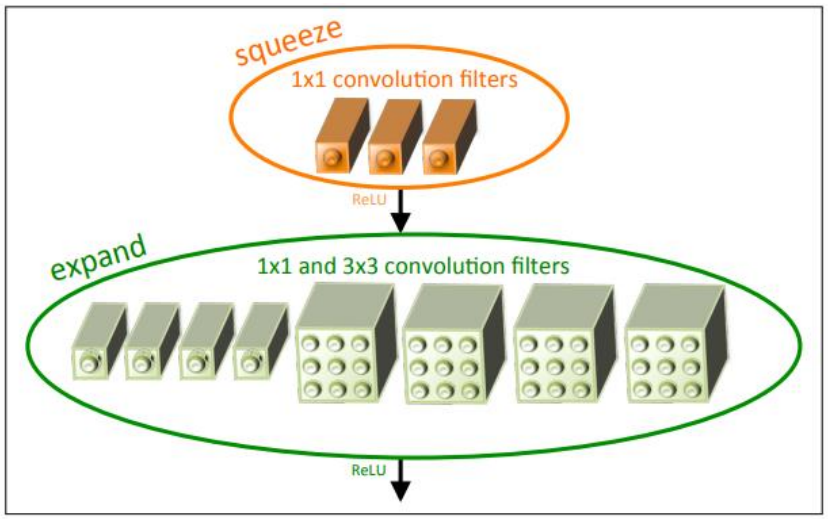



Figure 1.12 Fire module [21].

The aforementioned lightweight neural networks propose different methods to improve and optimize the architecture and computations of models. This enhances the feasibility of deploying model on mobile devices with restriced hardware resource.

# 1.3 Introduction to AIDER dataset and its related work

The creation of AIDER dataset entailed the manual collection of images for four disaster classes: Fire, Flood, Collapsed Building, Traffic Accidents, along with a category representing the Normal class, as shown in Figure 1.13.

Figure 1.13 The sample images from AIDER dataset [9].

In [22], the authors use a special convolution called atrous convolution (also known as dilated convolution) to acquire and adjust images at varying dilation rates, effectively increasing their receptive field to concurrently grasp features at different resolutions, then merge them to enable the model to learn from a broader receptive field without inflating the parameters, as depicted in Figure 1.14.

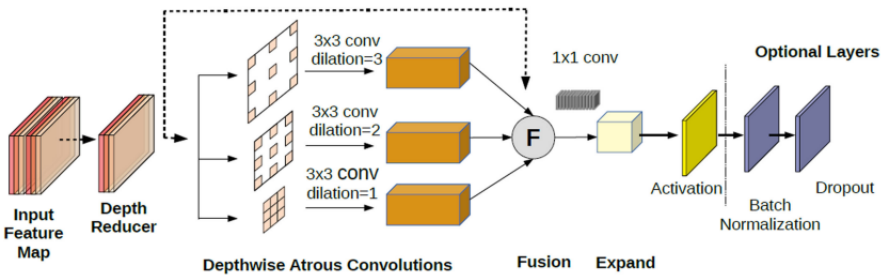



Figure 1.14 Atrous convolution block [22].

In [23], the author applies data augmentation techniques to the dataset, including rotation, flipping (horizontal and vertical), and slight adjustments to the image intensity, to improve the robustness of the model training. Additionally, the author investigates various design components of the model, such as layer configurations and types, to strike a balance between accuracy and model complexity.

In [24], the author uses EfficientNet as the backbone, employing the width principle as a basis to discover a shallow network while preserving the same performance and reducing computational complexity. Additionally, the author integrates the attention module [25], as depicted in Figure 1.15, which includes spatial attention and channel attention, to enable the network to concentrate on the most informative features within feature maps.

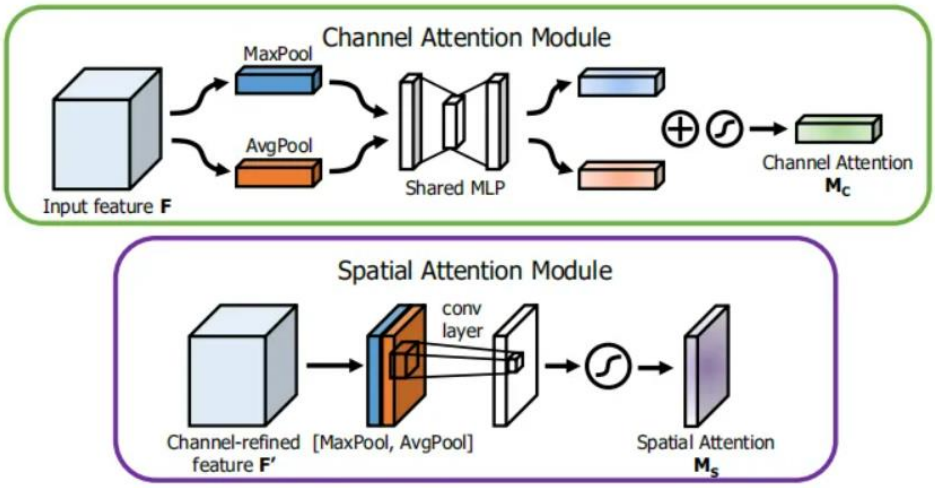



Figure 1.15 The modules of channel attention and spital attention [25].

However, contemporary methodologies often prioritize achieving high accuracy without considering the actual computational complexity and energy consumption, resulting in limited flight time for UAVs, preventing them from effectively performing disaster detection tasks. Therefore, our goal is to develop an energy-efficient and lightweight method for UAVs to rapidly detect and categorize disasters without compromising functionality and accuracy.

## 1.4 Branch Convolution Neural Network

Traditional Convolutional Neural Network (CNN) image classifiers usually follow a structure where subsequent convolution layers lead to an individual output layer. This design is founded upon the assumption that all target classes should be regarded with equal importance and distinctiveness. Nonetheless, it's recognized that certain classes may be more challenging to distinguish than others, and classes could be arranged hierarchically within categories. In [26], the author utilizes the softmax output of a DNN to categorize frequently confused categories. By averaging the softmax output across all input images belonging to various categories, it identifies categories that are commonly misclassified or confused with each other.

In [27], the author proposes a Hierarchical Deep Convolutional Neural Network (HD-CNN) to enhance classification accuracy. This is achieved by using a coarse category classifier to differentiate between easy classes, while employing fine category classifiers to differentiate complex classes. The coarse category classifier is trained to categorize images into one of the coarse categories, whereas the fine category classifiers are trained to classify images into one of the fine categories within each coarse category, as depicted in Figure 1.16. This method enables the use of specialized classifiers for challenging categories, resulting in enhanced accuracy.

Nonetheless, a potential challenge linked with the aforementioned approach is the need to pretrain both the coarse and fine category components, followed by a fine-tuning process, which could be time-consuming. Furthermore, HD-CNN employs only a single coarse category, leading to scalability concerns since there might be multiple levels of coarse categories in a hierarchical label tree for various datasets.

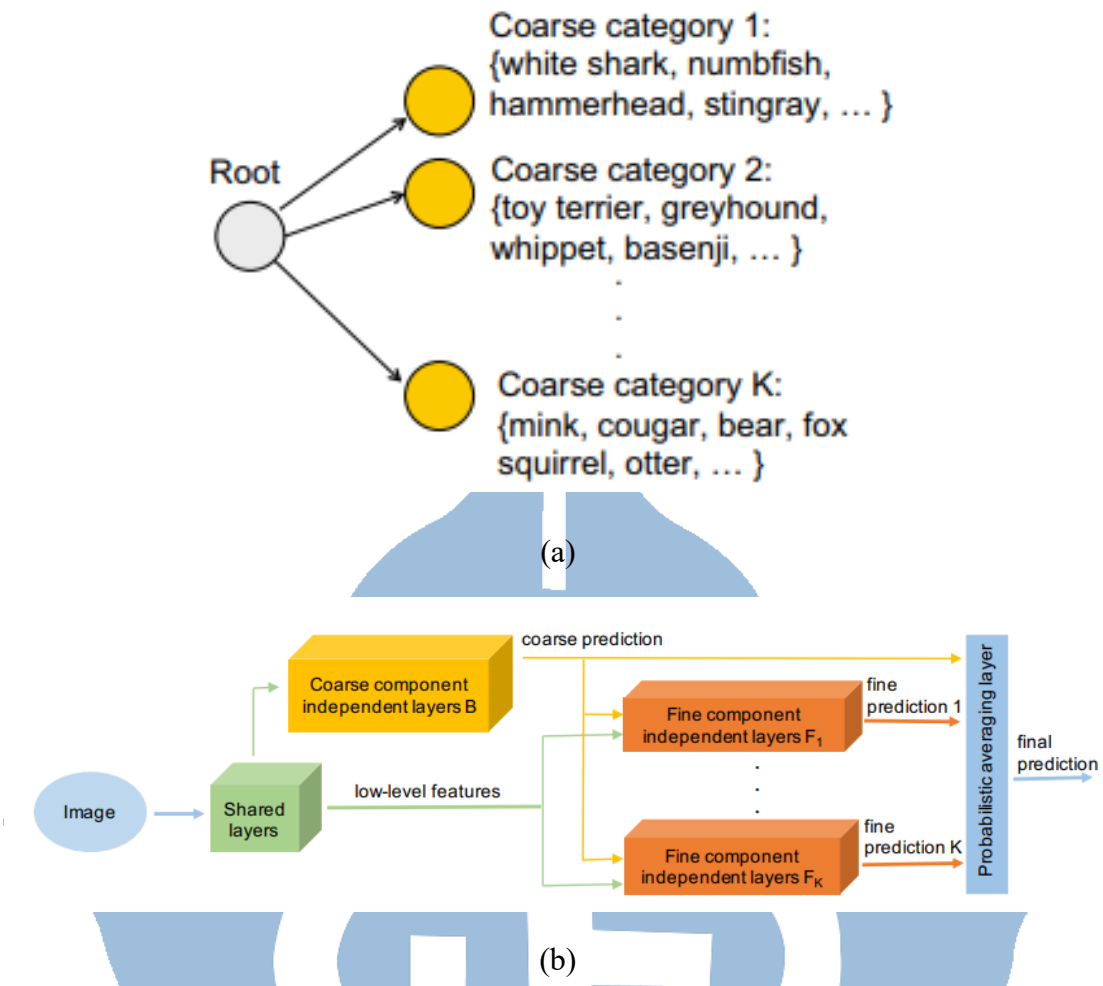



Figure 1.16 (a) A two-level category hierarchy (b) HD-CNN architecture [27].

Therefore, in [28], the Branch Convolutional Neural Network (B-CNN) is introduced, integrating serveral predictions arranged in a coarse-to-fine order across connected convolution layers. This arrangement aligns with the hierarchical form of the target categories, as depicted in Figure 1.17. This methodology is viewed as a way to integrate prior knowledge into the output. Through this approach, it has been shown that CNN-based models can be guided to progressively grasp from coarse-to-fine information. The integration of previous insights effectively improves the classification performance of CNN models.

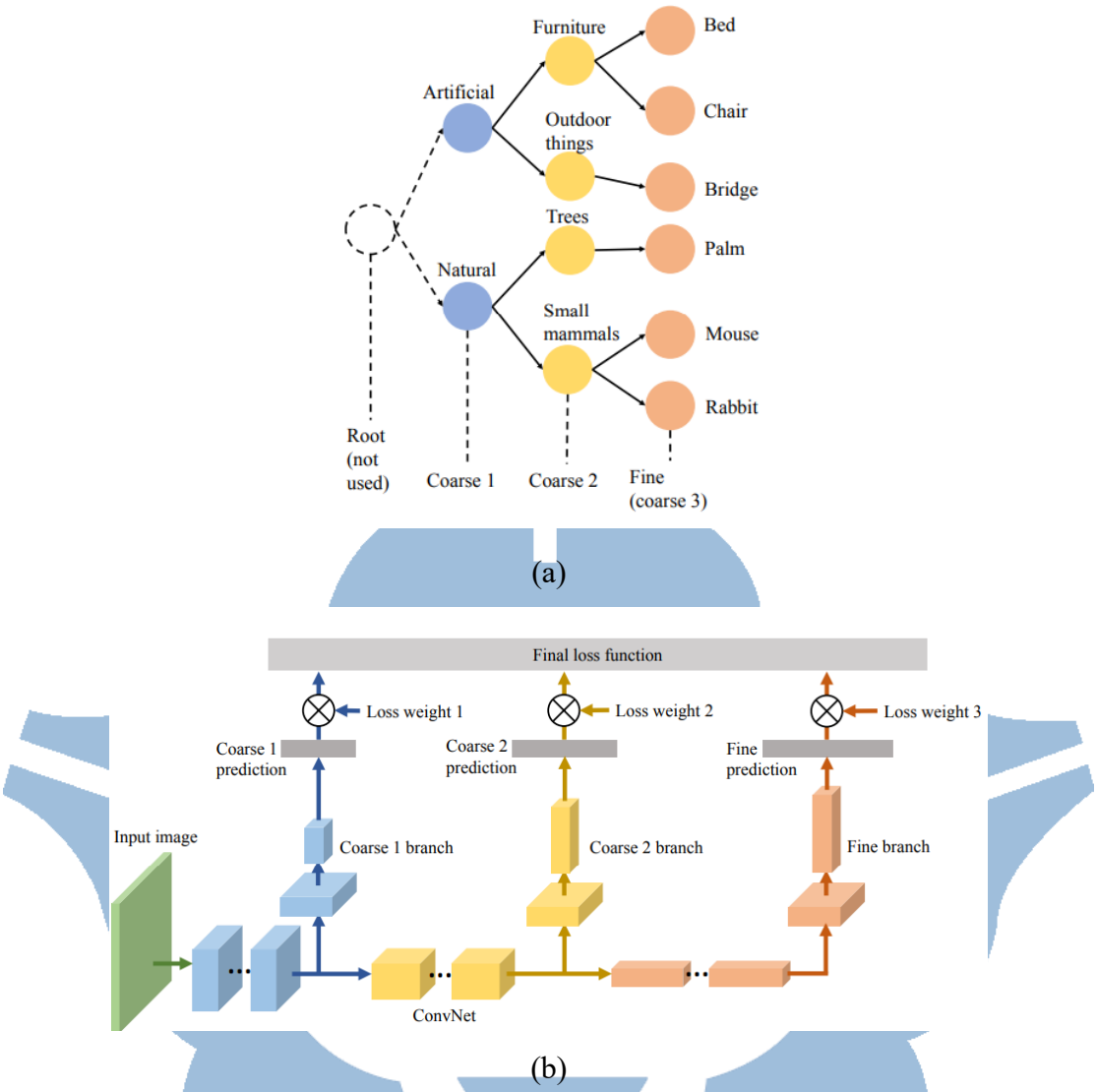



Figure 1.17 (a) Hierarchical label tree (b) B-CNN architecture [28].

In [29], the author introduces Hierarchical Neural Networks, where small deep neural networks are used at each level node. These networks are trained specifically for certain attributes, enabling quick identification of objects related to those attributes while reducing computational demands. As shown in Figure 1.18, although there is a slight drop in accuracy compared to state-of-the-art models, the memory usage and computation times are significantly reduced.

| Dataset | Technique | Model Size | FLOPs | Test Rank-1 | Test mAP |
|---|---|---|---|---|---|
| VRAI | RAM-VGG [18] | 528 | 15,483 M | 0.720 | 0.573 |
| | MultiTask [11] | 103 | 3,882 M | 0.685 | 0.693 |
| | MultiTask+DP [11] | 351 | 11,172 M | 0.803 | 0.786 |
| | DenseNet201 [8] | 77 | 4,340 M | 0.671 | 0.700 |
| | Random Tree | 25 | 2,026 M | 0.631 | 0.585 |
| | **Our Method** | 14 | 1,082 M | 0.781 | 0.737 |
| Market 1501 | Pyramidal [1] | 184 | 9,757 M | 0.928 | 0.821 |
| | DARE [15] | 89 | 2,891 M | 0.868 | 0.693 |
| | Auto-reID [6] | 55 | 2,050 M | 0.938 | 0.834 |
| | DG-Net [16] | 101 | 4,029 M | 0.896 | 0.745 |
| | ResNet50 [22] | 103 | 3,882 M | 0.872 | 0.685 |
| | DenseNet201 [8] | 77 | 4,000 M | 0.860 | 0.699 |
| | PCB [17] | 107 | 4,206 M | 0.923 | 0.774 |
| | Random Tree | 27 | 1,736 M | 0.788 | 0.535 |
| | **Our Method** | 14 | 808 M | 0.885 | 0.699 |

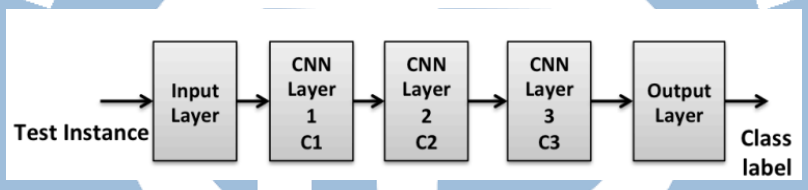
Figure 1.18 Comparisons with state-of-the-art [29].

Unlike traditional neural networks that must learn features for all categories, training for coarse and fine distinctions among different classes enhances the informativeness and interpretability of the model's predictions. This approach aids in feature learning and contributes to an overall improvement in accuracy.

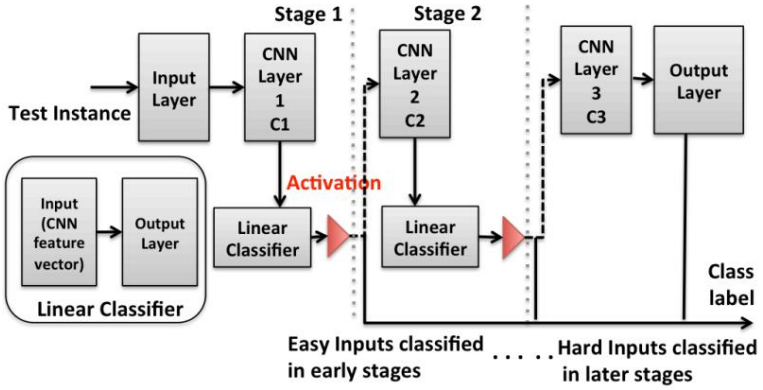
# 1.5 Early Exit Mechanism in Neural Network

Traditional CNN models excel in image recognition and object detection, possessing strong feature extraction capabilities and generalization performance. However, when dealing with large-scale image data or running on devices with limited computational resources, the inference process of traditional CNNs can become time-consuming and computationally expensive.

Therefore, [30] introduces Conditional Deep Learning (CDL) to selectively activate the deeper layers of the network based on the difficulty level of the input data, as shown in Figure 1.19. This adaptive mechanism allows the network to optimize its processing power and computational resources, thereby enhancing its overall performance.




(a)




(b)

Figure 1.19 (a) Traditional deep learning network (b) Conditional Deep Learning Network (CDLN) [30].

In [31], the author proposes BranchyNet, which utilizes side branch classifiers to determine when to exit the network early. When the model becomes confident enough to make predictions, it can speed up the process of stopping computations using the exit point, thus conserving time and computational power, as shown in Figure 1.20.

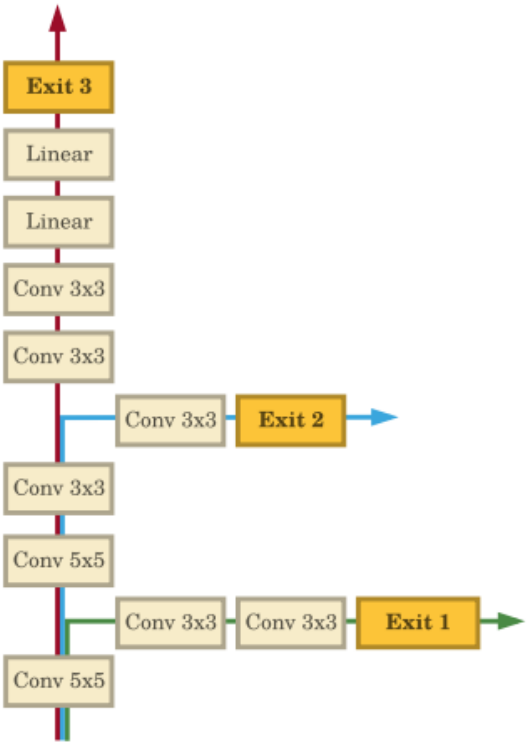


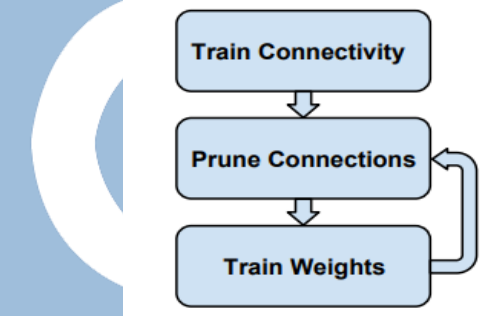

Figure 1.20 BranchyNet with two exit points added [31].

The use of neural networks with early exit mechanisms offers significant benefits in image recognition tasks. These advantages include conserving computational resources, enhancing inference speed, accommodating inputs of varying difficulty, and maintaining accuracy. By dynamically halting network operations during inference as required, the early exit mechanism enables the network to adapt the depth of computation, thereby enhancing efficiency without compromising accuracy. This presents a more efficient solution for practical applications, particularly when managing large image datasets or operating on devices with limited computational resources.

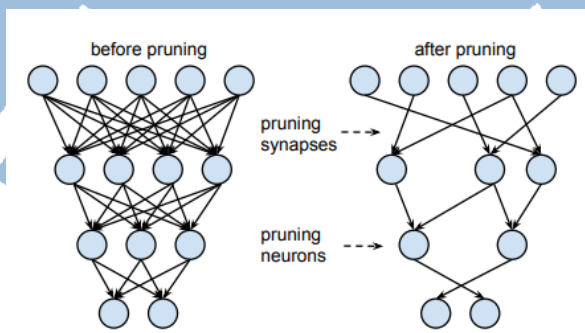
# 1.6 Model Compression in Neural Network

Model compression refers to a set of techniques and methodologies targeted at decreasing the computational workload and parameters of the model, making them more compact and efficient without significantly compromising their performance. The common techniques used in model compression include network pruning [32], weight sharing [33], knowledge distillation [34], and low-rank factorization [35].

Network pruning entails eliminating redundant neurons or connections from the network. This can be done during training (weight pruning) or after training (weight or neuron pruning). Pruned models have fewer parameters, leading to reduced storage and computation costs, as demonstrated in Figure 1.21.




(a)




(b)

Figure 1.21 (a) Three-step pruning (b) Pruned synapses and neurons before and after pruning [32].

Weight sharing focuses on minimizing storage requirements of weights. The central concept revolves around grouping weight values in the network and substituting each weight with the representative value of its corresponding cluster. This makes sure that weights within the same cluster have an representative value, as illustrated in Figure 1.22. By employing weight sharing, the model's memory demand is reduced, and computational requirements can be trimmed without substantially compromising performance.

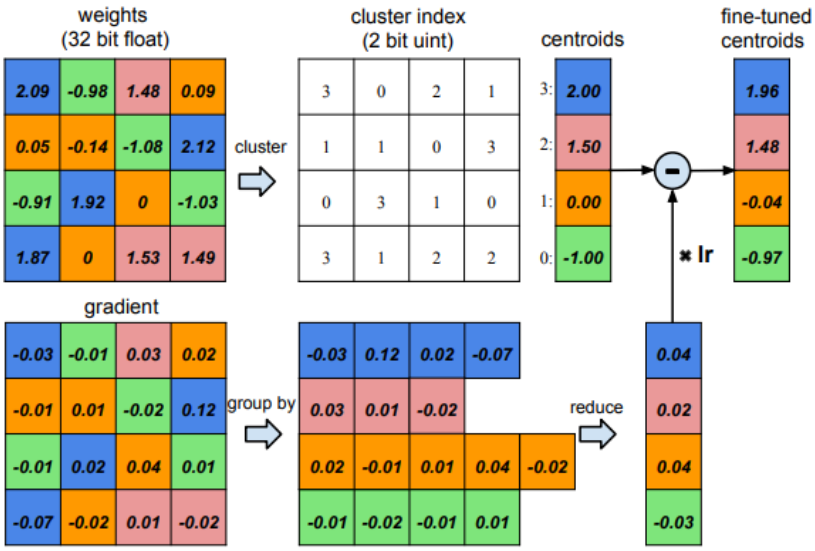


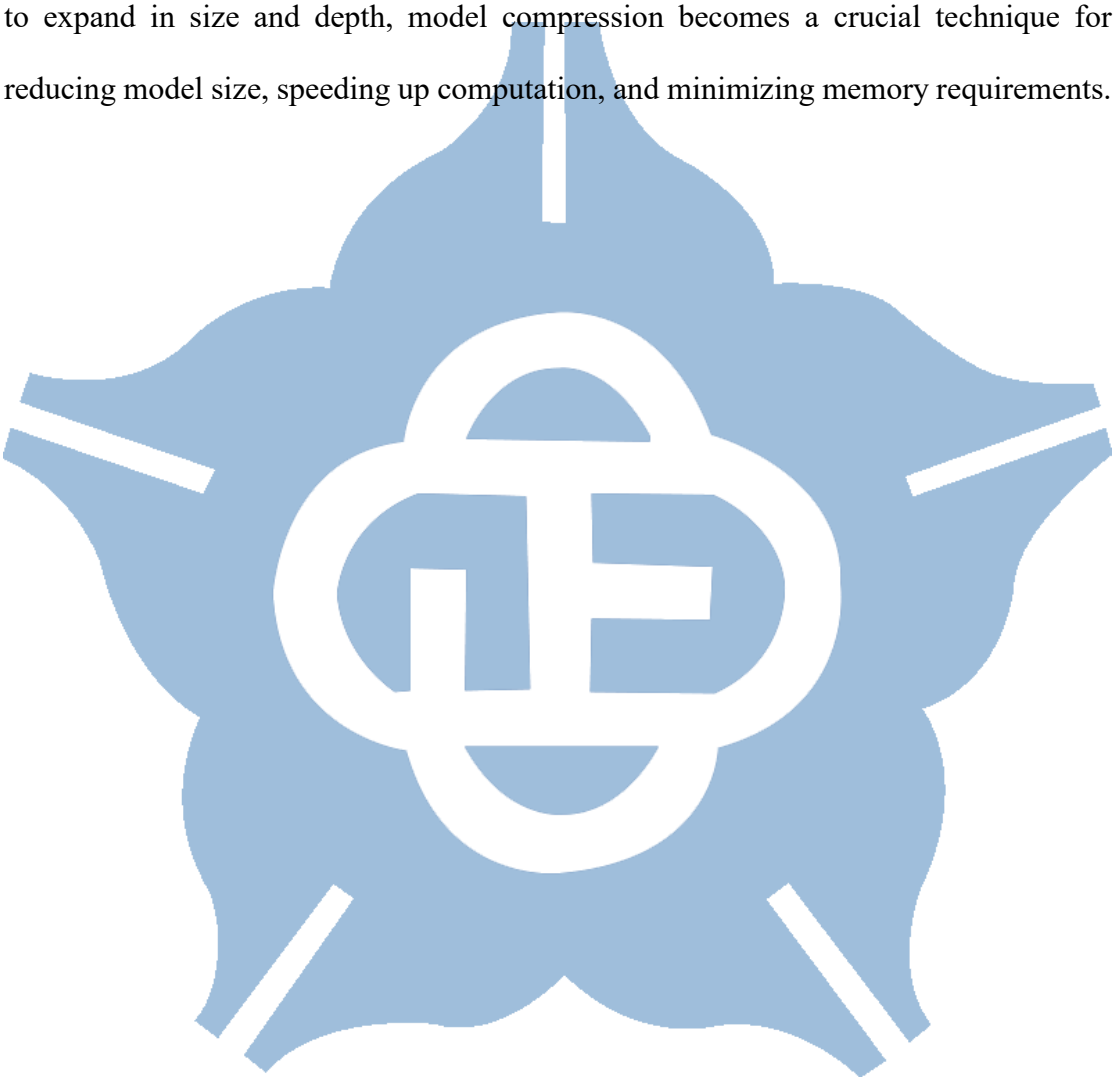

Figure 1.22 Weight sharing [33].

Knowledge distillation entails instructing a compact, lightweight model (referred to as the student) to emulate the functionality of a larger, more complicated model (referred to as the teacher). The student model is trained not only to replicate the teacher's predictions but also to learn from the softer, probabilistic outputs of the teacher, allowing for a more compact representation of knowledge.

Low-rank factorization is a mathematical technique used to represent a matrix by approximating it with a matrix of lower rank. It decomposing a matrix into three matrices with lower ranks using SVD (Singular Value Decomposition), thus decreasing

the number of parameters needed to indicate the original matrix. This technique is effective when the data in the matrix has inherent patterns or correlations that can be captured with fewer dimensions, making it more computationally efficient.

Exploring alternative model compression techniques beyond the commonly employed methods discussed earlier is worthwhile. As deep neural networks continue to expand in size and depth, model compression becomes a crucial technique for reducing model size, speeding up computation, and minimizing memory requirements.



# 1.7 Quantization Methods in Neural Network

Traditionally, training and inference of deep neural networks have relied on 32-bit floating-point numbers as computational units. While this approach effectively enhances model training and prediction accuracy, it also increases computational time and workload, especially when deployed on edge computing devices with limited hardware resources. Hence, quantization is the process of representing the model's parameters such as weights and activations in lower bit precision, reducing computational complexity and storage space while preserving prediction accuracy to a satisfactory level, thus enhancing the overall performance of the model.

Binary Neural Network (BNN) [36] is a type of neural network in which the weights of the network are constrained to binary values, typically +1 or -1, as shown in Equation 1.8. While BNN provides benefits in computational speed and storage space, they also come with challenges, such as reduced expressiveness and potential loss of accuracy compared to traditional networks with higher precision weights.

$$x_b = sign(x) = \begin{cases} 1, & if\ x > 0 \\ -1, & otherwise \end{cases} \tag{1.8}$$

Ternary Weight Network (TNN) [37] constrains the weight to one of three values: +1, 0, or -1, as shown in Equation 1.9. This ternary representation provides a compromise between the binary weights of BNN. Ternary weights allow for a reduction in both computational overhead and model size while maintaining a certain level of precision compared to binary weights.

$$w_i^t = \begin{cases} +1, if\ x > \Delta \\ 0, if\ x \leq \Delta \\ -1, if\ x < 0 \end{cases} \tag{1.9}$$

While BNN and TWN can effectively reduce computational complexity and

model memory requirements by decreasing the bit precision to 1-bit and 2-bit, respectively, there still exists some gap in their prediction accuracy compared to full precision models.

LQ-Nets [38] trains a quantized deep neural network alongside its corresponding quantizers, enabling the network to operate efficiently with reduced precision. This approach allows for the optimization of both weight and activation quantization, thereby improving the model's performance while retaining its compactness.

The Wide Reduced-Precision Networks (WRPN) [39] reduces the precision of activation maps (along with model parameters) and increases the number of filter maps in a layer to compensates for the loss of information induced by reducing the precision and allows for reduced-precision activations without sacrificing model accuracy.

The widely used activation function is ReLU in various neural networks due to its simple and efficient characteristics. However, its unbounded output values result in the need for a sufficient precision range when quantizing its activation values for accurate representation. Although we can clip the values to limit the output range, determining an optimal clip value is challenging due to differences in the structure between models and even between layers within a model. Therefore, the parameterized clipping activation for quantized neural networks (PACT) [40] substitute ReLU function with an activation function featuring a parameterized clipping level, denoted as α. This α value is dynamically adjusted through back-propagation during training to determine the appropriate quantization scale. This technique strategically sets the quantization scale to be smaller than that of ReLU, effectively mitigating the quantization error. The activation function is determined by Equation 1.10.

$$y = PACT(x) = 0.5(|x| - |x - \alpha| + \alpha) = \begin{cases} 0, x \in (-\infty, 0) \\ x, x \in [0, \alpha) \\ \alpha, x \in [\alpha, +\infty] \end{cases} \quad (1.10)$$

DoReFa-Net [41] introduces the capability to quantize weights and activations to any bit width, addressing the limitation of insufficient quantization flexibility observed in BNN and TWN. The quantization equation is expressed in Equation 1.11, which quantizes the input number x into a k-bit representation. The equation for low-bit width weight quantization is defined in Equation 1.12, where the max function is applied over all weights within the same layer. The tanh function is employed to constrain the numerical range of the weight to [-1, 1], ensuring that the quantized weight represents a k-bit fixed-point number within this range. For activation quantization, a bounded function needs to be applied to ensure that the input activations x are scaled to the range [0,1], as demonstrated in Equation 1.13. Afterward, the input number x is quantized into k-bit using Equation 1.14.

$$quantized_k(x) = \frac{1}{2^k-1} round\Big((2^k-1)x\Big) \tag{1.11}$$

$$f_w^k(r) = 2quantize_k\left(\frac{\tanh(r)}{2\max(|\tanh(r)|)} + \frac{1}{2}\right) - 1 \tag{1.12}$$

$$h(x) = clip(x, 0, 1) \tag{1.13}$$

$$f_a^k(r) = quantize_k(r) \tag{1.14}$$

Obviously, The quantized neural networks can reduce computational resource demands and improving computational speed. However, it is crucial to note that quantization techniques can also increase errors to the model's predictions. Therefore, deciding on an appropriate quantization method involves considering various factors and requirements, including the specific characteristics of the task, the type of model architecture, and the trade-off between model complexity and accuracy.

## 1.8 Motivation

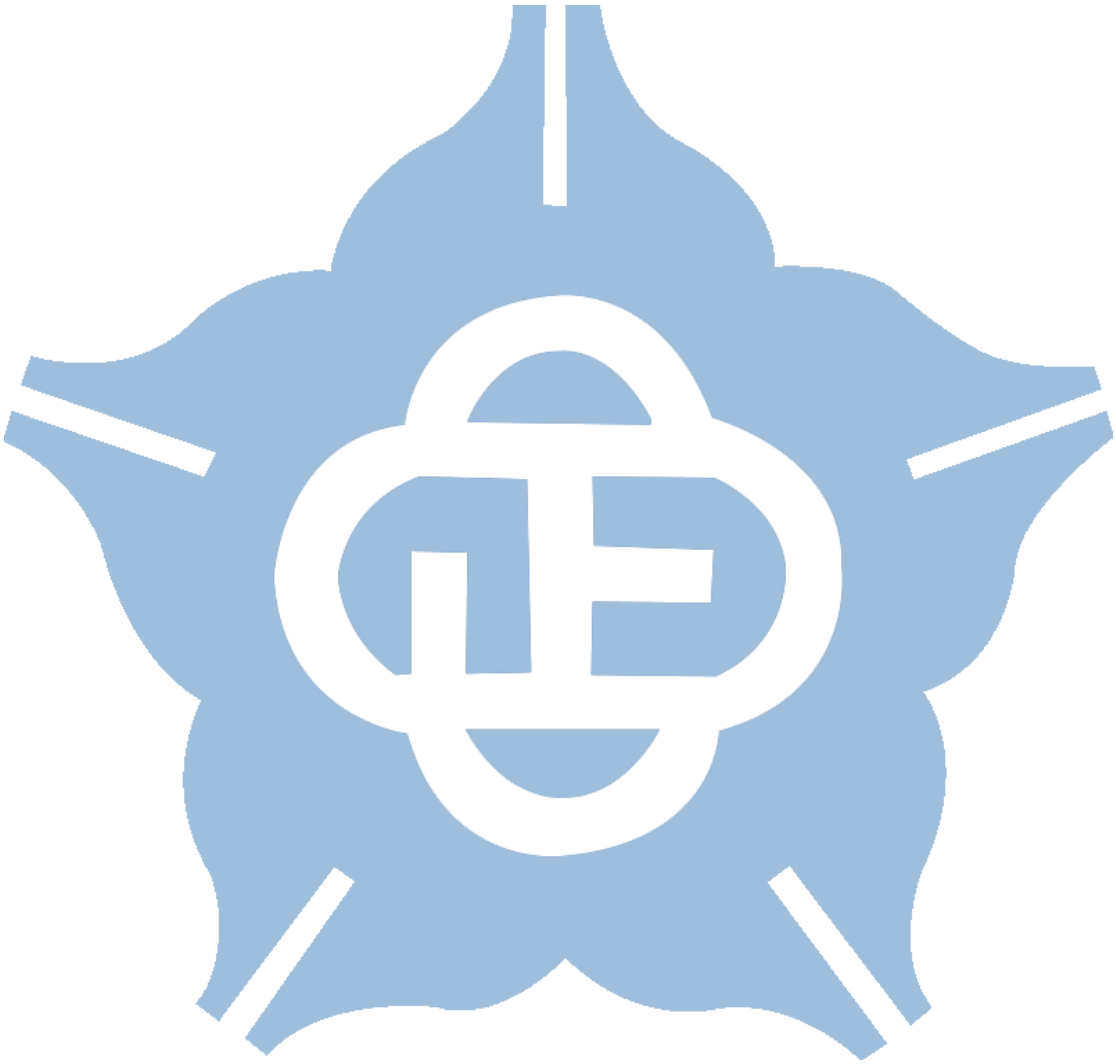
Throughout the previous sections, we have analyzed and discussed a variety of disaster detection methods with drones. The integration of deep-learning models can substantially improve search-and-rescue missions and disaster management efforts. Nevertheless, due to the power consumption and hardware resources of drones, the extensive computations required by deep neural network present a significant challenge. Hence, it is necessary to find and explore approaches that strike a balance between model size and classification accuracy.

In this thesis, we design a lightweight CNN model for disaster recognition. A hierarchical classification is integrated into the model with the concepts of coarse and fine to boost the prediction accuracy. Moreover, the utilization of an early exit mechanism reduces the computational cost significantly. For hardware implementation and develpoement on a 16-nm CMOS process, weight and activation quantization techniques are employed to decrease the demand for memory space. Therefore, our research has made it more feasible and effective to deploy neural networks on drones for disaster classification tasks.

The main contributions of this work are summarized as follows:

1. The integration of hierarchical classification with coarse and fine prediction improves the informativeness and interpretability of the learned features. This approach can significantly increase accuracy, thereby achieving the same model performance with fewer parameters.
2. The development of a lightweight CNN model with early exit mechanism efficiently diminishes the computational burden of the neural network, thereby enabling fast inference and power-saving during disaster recognition.
3. The utilization of DoReFa-Net quantization techniques lower the memory

requirements for weights storage. This accomplishment illustrates the feasibility of reducing computational resources while maintaining model performance.

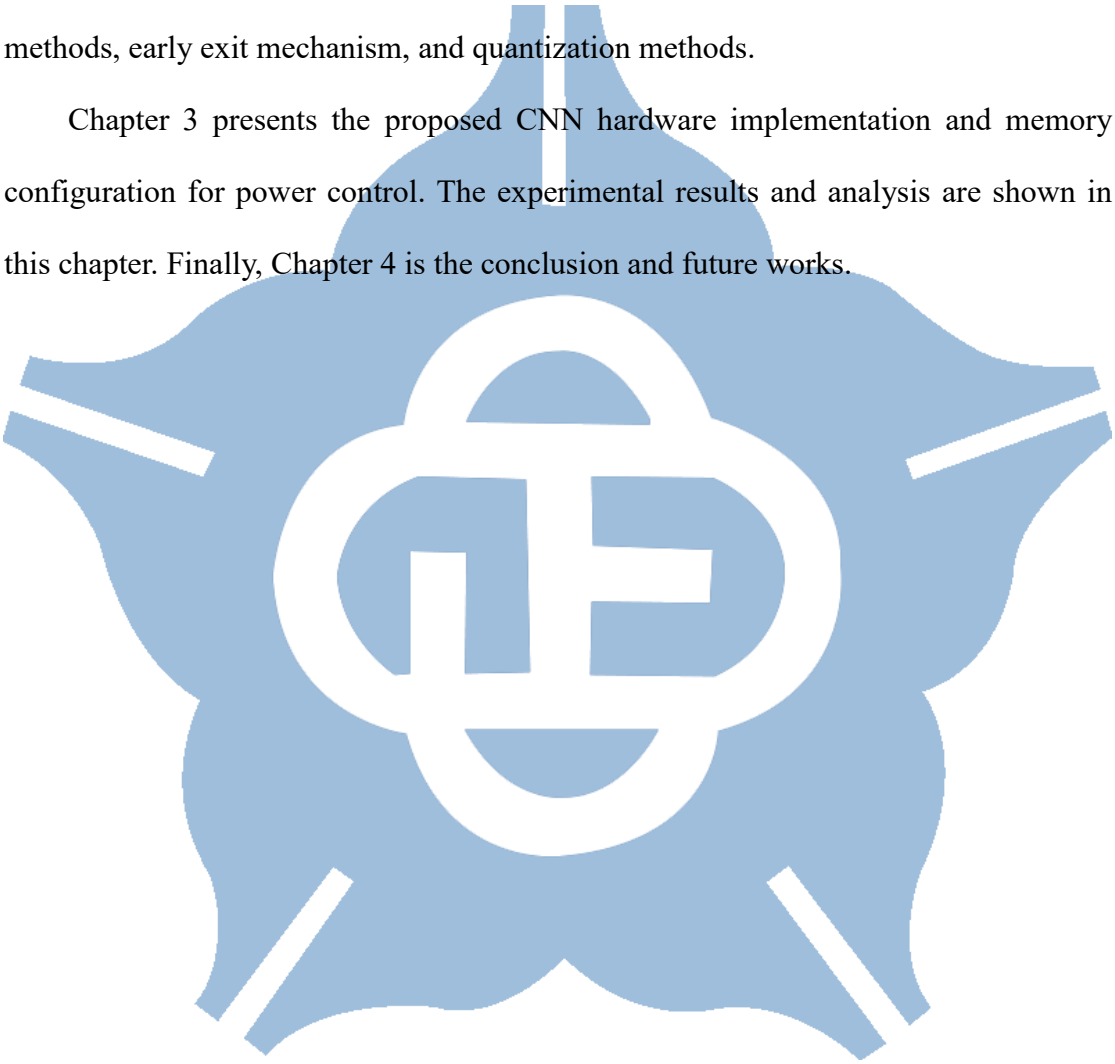


## 1.9 Thesis Organization

Chapter 1 introduces relevant papers and datasets on UAV disaster detection. It also mentions common CNN architectures and B-CNN, followed by a discussion on hardware-friendly quantization methods.

Chapter 2 presents the proposed CNN software architecture, dataset preprocessing methods, early exit mechanism, and quantization methods.

Chapter 3 presents the proposed CNN hardware implementation and memory configuration for power control. The experimental results and analysis are shown in this chapter. Finally, Chapter 4 is the conclusion and future works.



# Chapter 2 Software Implementation

## 2.1 Architecture Overview

In this thesis, an approach for disaster detection using UAVs is proposed. As mentioned in [9], the dataset consists of four disaster events: Fire, Flood, Collapsed building, Traffic accidents, along with one category representing the Normal class, as shown in Figure 1.14. A total of 6,433 aerial images were obtained, after shuffling the dataset, 80% are used to train the model, and 20% are used to test the model, the detailed information of training set and test set in each class is illustrated in Table 2.1.

Table 2.1 The amount of training set and test set in each class.

| **Class** | **Amount of training data** | **Amount of test data** |
|---|---|---|
| Fire | 417 | 104 |
| Flood | 421 | 105 |
| Collapsed building | 409 | 102 |
| Traffic accidents | 388 | 97 |
| Normal | 3,512 | 878 |
| **Total** | 5,147 | 1,286 |

The flowchart for constructing the branch CNN model with early exit is depicted in Figure 2.1. After preprocessing the image, the first step is to create the label tree for the corresponding dataset, which will group frequently confused classes together for hierarchical prediction. Once the data preprocessing and label tree are done, then an appropriate neural network architecture is chosen after several trials. Subsequently, due

to the images captured by drone are typically normal class in most cases, early exit mechanism through coarse prediction can be applied to save energy and reduce computational costs. Next, Keras is utilized for constructing the model and training the CNN model until it attains sufficient test accuracy.

Once the model is built, the subsequent step is to perform quantization on the model. Considering the limited processing capabilities of edge devices, conserving storage capacity and minimizing the model's computational load are essential. Nonetheless, quantizing the model often results in a decrease in accuracy, so finding a balance between accuracy and computational complexity becomes a significant concern. Finally, using Python code to validate the consistency between software and hardware computation methods, then concluding the software implementation phase.

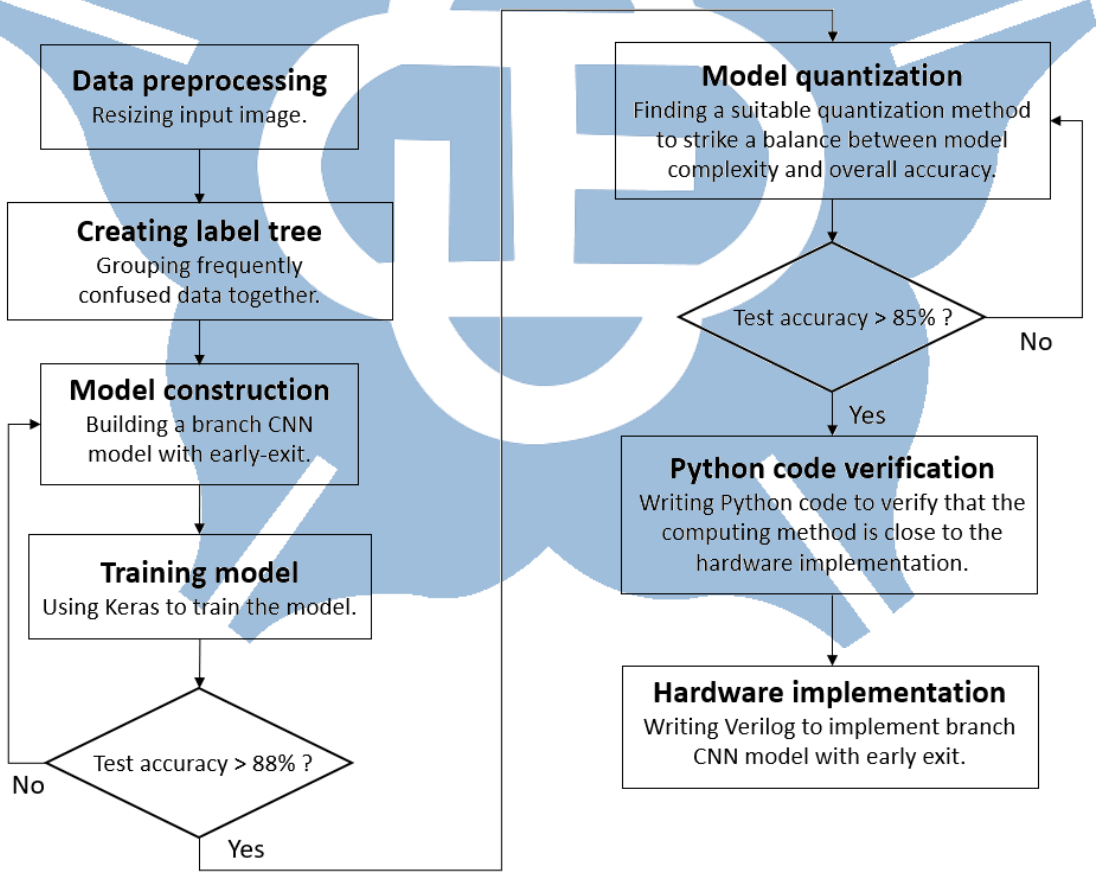



Figure 2.1 The flow chart for building a branch CNN model.

## 2.2 Dataset Preprocessing

When considering a suitable image preprocessing approach, it's crucial to evaluate its potential impact on model accuracy and resource consumption during image processing. Since the images in the dataset do not have a fixed size, adjustments to the image size will begin with resizing to 128×128 pixels, taking into account model complexity and accuracy. Initially, a five-layer CNN model is established to evaluate various methods. Subsequently, in Table 2.2, separate experiments are conducted using these compression methods to assess the impact of different image sizes on accuracy.

The findings in Table 2.2 show the major influence of the nearest-neighbor interpolation method on precision, revealing a significant drop when scaling images to 32×32 pixels. Conversely, the other three interpolation methods exhibit better performance. Although the performance of area and bicubic interpolation methods is slightly better than bilinear, their computational complexity is much higher than bilinear. Therefore, when taking computational complexity and accuracy into account, the proposed architecture adopts the bilinear interpolation method, resizing images to 64×64 pixels for subsequent model training and testing.

Table 2.2 The accuracy of different compression methods across various image sizes.

| **Image size** | **Nearest neighbor interpolation** | **Bilinear interpolation** | **Area interpolation** | **Bicubic interpolation** |
|---|---|---|---|---|
| 128 × 128 | 85.31% | 90.28% | 90.34% | 90.31% |
| 64 × 64 | 78.72% | 90.12% | 90.21% | 90.25% |
| 32 × 32 | 66.34% | 73.57% | 74.39% | 75.44% |

## 2.3 Implementation of Branch CNN and Software Analysis

As discussed in Section 1.4, before building the branch CNN model, establishing a label tree for the dataset is necessary, which involves categorizing classes that are frequently confused. The dataset used consists of a total of five classes, with the Normal class being separated to facilitate its inclusion in an early exit mechanism. Initially, a compact neural network consisting of five layers is constructed, followed by training the model using the remaining classes intended for classification. From Table 2.3, it is evident that the class with the highest number of judgments related to Traffic accidents is Collapsed building, indicating a tendency for these two classes to be confused. Consequently, Traffic accidents and Collapsed building are grouped together, forming a subgroup named Accident, as shown in Figure 2.2. Similarly, it can be observed that Fire and Flood have the highest number of judgments related to each other. Therefore, these two classes are grouped together, forming a subgroup named Disaster, as shown in Figure 2.3.

Table 2.3 The confusion matrix for creating the label tree.

| | **Fire** | **Flood** | **Collapsed building** | **Traffic accidents** |
|---|---|---|---|---|
| **Fire** | **75** | **25** | 3 | 1 |
| **Flood** | **25** | **67** | 6 | 7 |
| **Collapsed building** | 6 | 11 | **47** | **38** |
| **Traffic accidents** | 8 | 6 | **27** | **56** |

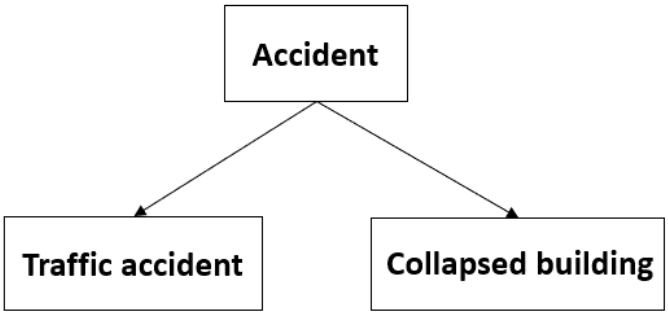



Figure 2.2 The subcategory Accident grouped by Traffic accidents and Collapsed building.

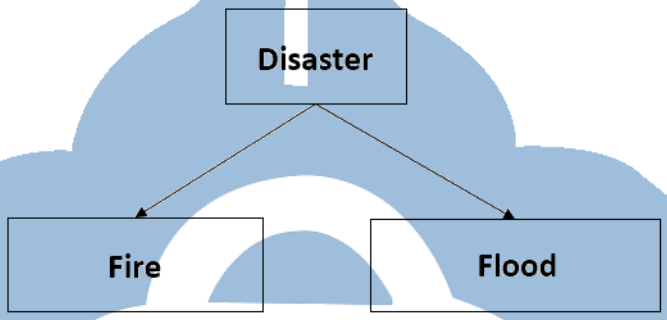



Figure 2.3 The subcategory Disaster grouped by Fire and Flood.

Finally, the corresponding label tree for the dataset has been established, as shown in Figure 2.4. Therefore, the branch CNN model will generate two types of predictions for each input image corresponding to the label tree, referred to as coarse and fine predictions. For example, when dealing with an image of Fire, the branch CNN model will first produce a coarse output of Disaster, and then subsequently generate a fine output of Fire. This strategy enhances the informativeness and interpretability of the model's predictions, facilitates feature learning, and ultimately leads to a general improvement in accuracy. Hence, the same model performance can be achieved using fewer parameters.

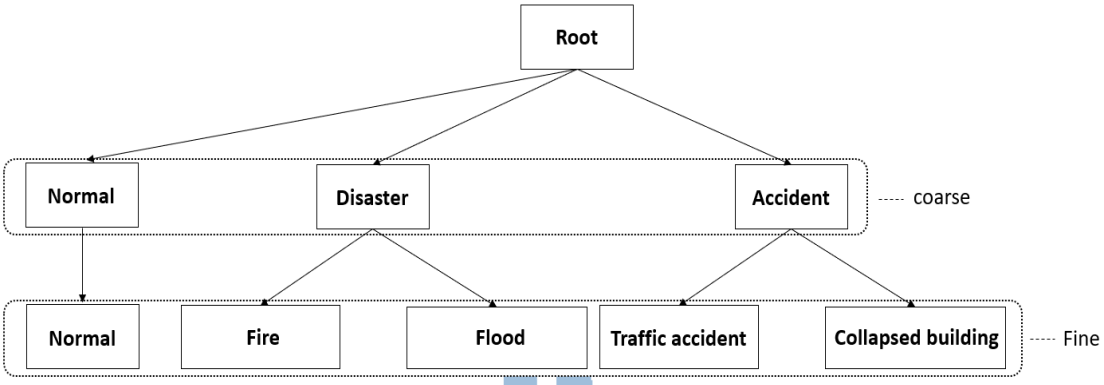



Figure 2.4 The label tree corresponding to the dataset.

This section provides a detailed explanation of the proposed branch CNN architecture. The size of feature map for each layer in proposed branch CNN model are presented in Table 2.4. To preserve the feature map size after the convolution operation, zero padding is applied. However, zero padding may lead to the covering of edge data points by the receptive field center of the kernel. The convolution operation uses a stride size of 1, allowing the kernel to encompass more functions, albeit increasing computational load. As a result, the pooling operation adopts a stride of 2 to reduce the size of subsequent feature maps, thereby reducing memory requirements and computational costs.

Table 2.4 The size of input and output data in each layer.

| **Operation** | **Input data size** (width × height × in_channel) | **Output data size** (width × height × in_channel) | **Stride** |
|---|---|---|---|
| Convolution 1 | 64 × 64 × 3 | 64 × 64 × 8 | 1 |
| Max-pooling 1 | 64 × 64 × 8 | 32 × 32 × 8 | 2 |
| Convolution 2 | 32 × 32 × 8 | 32 × 32 × 16 | 1 |
| Max-pooling 2 | 32 × 32 × 16 | 16 × 16 × 16 | 2 |
| Convolution 3 | 16 × 16 × 16 | 16 × 16 × 24 | 1 |
| Max-pooling 3 | 16 × 16 × 24 | 8 × 8 × 24 | 2 |

| | | | |
|---|---|---|---|
| Global average pooing | 8 × 8 × 24 | 1 × 1 × 24 | - |
| Fully-connected | 1 × 1 × 24 | 1 × 1 × 3 | - |
| Convolution 4 | 8 × 8 × 24 | 8 × 8 × 32 | 1 |
| Max-pooling 4 | 8 × 8 × 32 | 4 × 4 × 32 | 2 |
| Convolution 5 | 4 × 4 × 32 | 4 × 4 × 40 | 1 |
| Max-pooling 5 | 4 × 4 × 40 | 2 × 2 × 40 | 2 |
| Global average pooling | 2 × 2 × 40 | 1 × 1 × 40 | - |
| Fully-connected | 1 × 1 × 40 | 1 × 1 × 5 | - |

Additionally, after the convolution operation, batch normalization is conducted to enable the model to learn data regularities. Following this, the activation function is applied to introduce nonlinear properties into the model. ReLU is the activation function used in the proposed CNN model. It was selected due to its straightforward computational process and high convergence speed, which aids in reducing the model's training time. The complete operation steps for each layer are illustrated in Figure 2.5.


Figure 2.5 The entire operation in each layer.

After the operation of convolution is finished, global average pooling is chosen instead of using multiple fully connected layers. This helps in reducing the number of parameters, mitigating the risks of overfitting, and enhancing computational efficiency. However, it is still necessary to retain one fully connected layer for making model predictions, as shown in Figure 2.6.


Figure 2.6 The entire operation in each classifier.

Table 2.5 compares the test accuracy of various methods with different numbers of layers and channels to select the optimal model architecture. Based on the experimental results, attaining satisfactory accuracy necessitates increasing the number of channels when the layers are set to 4. However, boosting the channels lead to a notable escalation in computational expenses, thereby increasing model complexity and inference time. Conversely, with the layers set to 5, it becomes feasible to diminish the channels while maintaining the same model performance. Despite Method 2 slightly outperforming Method 3 in accuracy, its computational overhead is tripled. Hence, the proposed branch CNN model adopts Method 3 for subsequent research and analysis.

Table 2.5 The test accuracy of the branch CNN model with various layers and channels.

| **Method** | **Conv. layer (in_channel/out_channel)** | | | | | **FLOPs** | **Accuracy** |
|---|---|---|---|---|---|---|---|
| | **Layer1** | **Layer2** | **Layer3** | **Layer4** | **Layer5** | | |
| 1 | 3/8 | 8/16 | 16/32 | 32/64 | - | 9.1M | 90.57% |
| 2 | 3/8 | 8/32 | 32/64 | 64/64 | - | 21M | 91.13% |
| ***3*** | ***3/8*** | ***8/16*** | ***16/24*** | ***24/32*** | ***32/40*** | ***7.4M*** | ***90.21%*** |
| 4 | 3/8 | 8/16 | 16/32 | 32/32 | 32/64 | 8.5M | 90.14% |

Furthermore, Table 2.6 illustrates the discrepancy in accuracy between the branch CNN model with and without batch normalization. Although batch normalization increases hardware burden by introducing additional parameters, it has significant advantages in improving model training speed and stability. With batch normalization,

the accuracy increases by approximately 3 to 4%. Hence, incorporating batch normalization into the proposed model becomes essential. This adjustment is particularly important for maintaining consistent performance across various training conditions.

Table 2.6 The test accuracy of the branch CNN model with and without BN.

| | **With batch normalization** | **Without batch normalization** |
|---|---|---|
| **Test accuracy** | **90.21%** | 86.67% |

Moreover, it is essential to examine and analyze the impact of placing the coarse classifier in the proposed model on the overall accuracy. As depicted in Figure 2.7, a total of four positions will be tested, denoted as Test1 to Test4. From Table 2.7, it can be seen that due to the fewer layers and kernel numbers in the model, the classification performance of the coarse classifier at Test1 and Test2 is not as anticipated, resulting in relatively lower accuracy of the fine classifier. Although Test4 involves more layers and kernels, it is closer to the fine classifier, causing the coarse classifier to fail to adjust the model promptly through loss weight, leading to a slightly lower accuracy of the fine classifier compared to Test3. Hence, considering the overall enhancement in accuracy, the proposed model positions the coarse classifier at Test3.

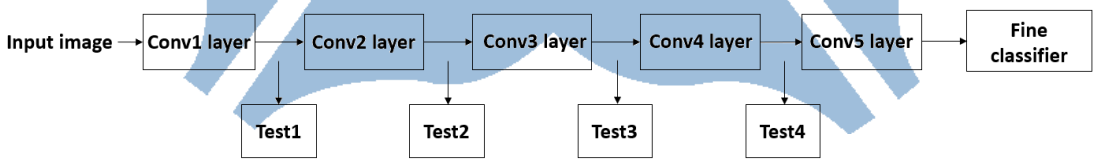



Figure 2.7 The test placement of coarse classifier in proposed model.

Table 2.7 The test accuracy of the placement of coarse classifier in proposed model.

| | **Test1** | **Test2** | **Test3** | **Test4** |
|---|---|---|---|---|
| **Test accuracy** | 85.57% | 87.85% | **90.21%** | 90.07% |

Additionally, experiments were conducted on three different B-CNN architectures: (1) The model is divided into two networks and does not include a label tree: a 5-layer model for all classes and a 3-layer model for the Normal class, (2) The model is divided into two networks and include a label tree: a 5-layer model for all classes and a 3-layer model for the Normal class, and (3) a 5-layer model with a label tree (current architecture). As shown in Table 2.8, architecture (1) did not establish a label tree for branch training, resulting in lower accuracy compared to (2) and (3). While architecture (2) includes a label tree for branch training, it consists of two separate networks, leading to higher computational and parameter requirements than (3). Therefore, by integrating a hierarchical structure of target classes into the model, branch CNN model gains advantages from structured prior knowledge that guides the classification process compared to traditional CNN models.

Table 2.8 The comparisons of different B-CNN architectures.

| | **Accuracy** | **Parameters** | **FLOPs** |
|---|---|---|---|
| (1) | 86.72% | 29,160 | 13.5M |
| (2) | 90.15% | 29,160 | 13.5M |
| **(3)** | **90.21%** | **24,008** | **7.4M** |

The total number of parameters in the entire neural network model is provided in Table 2.9. The parameters for convolutional layers and batch normalization can be computed using the equations given in Equations 2.1 and 2.2, respectively. Equation 2.3 presents the formula for fully connected layers. Following global average pooling, there is a notable reduction in the model's parameter count. Summing up all these parameters gives us the total number of parameters in the proposed model. In this thesis, the branch CNN model on the software platform comprises approximately 24,000 parameters in total.

$$Conv\ parameters = in\_channel \times kernel\ size \times out\_channel \quad (2.1)$$

$$BN\ parameters = 4 \times kernel\ size \times out\_channel \quad (2.2)$$

$$FC\ parameters = feature\ map\ size \times out\_channel \quad (2.3)$$

Table 2.9 The total parameters of B-CNN model.

| **Operation** | **Number of parameters** | **Sum of parameters** |
|---|---|---|
| Convolution 1 | 3 × 3 × 3 × 8 | 216 |
| Batch Normalization 1 | 4 × 8 | 32 |
| Convolution 2 | 8 ×3 × 3 × 16 | 1,152 |
| Batch Normalization 2 | 4 × 16 | 64 |
| Convolution 3 | 16 × 3 × 3 × 24 | 3,456 |
| Batch Normalization 3 | 4 × 24 | 96 |
| Fully connected | 24× 3 | 72 |
| Convolution 4 | 24 × 3 × 3 × 32 | 6,912 |
| Batch Normalization 4 | 4 × 32 | 128 |
| Convolution 5 | 32 × 3 × 3 × 40 | 11,520 |
| Batch Normalization 5 | 4 × 40 | 160 |
| Fully connected | 40 × 5 | 200 |
| **Total parameters:** | | 24,008 |

# 2.4 Branch CNN with Early Exit Mechanism

In most cases, images captured by drones belong to the Normal class. Therefore, early exit for the Normal class can be implemented in the proposed model's coarse classifier, as shown in Figure 2.8. The benefits of early exit mechanisms have been discussed in Section 1.5.

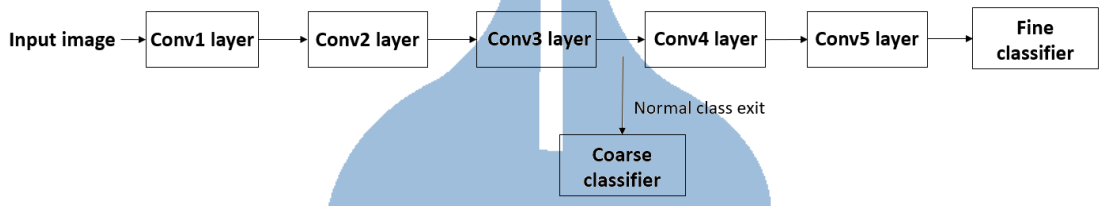



Figure 2.8 The early exit of Normal class through coarse classifier in proposed model.

From Table 2.10, it can be observed that the accuracy of the coarse classifier for the Normal class is lower than that of the fine classifier, but it can still maintain an accuracy of over 85%. However, the model can achieve detection with fewer parameters and computational load in the coarse classifier.

Table 2.10 The comparison of coarse and fine classifer to the Normal class.

| | **Parameters** | **FLOPs** | **Accuracy** |
|---|---|---|---|
| **Coarse classifer** | 5,152 | 6.1M | 88.67% |
| **Fine classifier** | 24,008 | 7.4M | 95.32% |

Therefore, the proposed branch CNN model can achieve fast inference and energy saving by implementing early exit for the Normal class through the coarse classifier, while predicting other disaster or accident classes with high accuracy using the fine classifier, allowing drones with limited hardware resouces can execute disaster detection more efficiently.

## 2.5 Weight Quantization Method

According to Table 2.9, there are 23k convolution parameters. Hence, quantizing the convolution parameters is the most practical method for effectively reducing the memory requirements and computational costs.

Firstly, the proposed architecture employs binary neural network (BNN) and ternary weight network (TWN) to quantize weights because these methods can significantly reduce the weights to 1 bit and 2 bits, as shown in Equation 1.8 and Equation 1.9. However, from Table 2.11, it is evident that the accuracy of both the coarse classifier and the fine classifier experiences a noticeable decline, indicating a gap from practical application.

Table 2.11 The accuracy of BNN and TWN.

| | **BNN** | **TWN** |
|---|---|---|
| **Coarse accuracy** | 64.48% | 72.83% |
| **Fine accuracy** | 67.02% | 75.14% |

Unlike BNN and TWN, which can only quantize weights into 1 bit and 2 bits, DoReFa-Net offers a quantization method that allows for flexible adjustment of bit numbers. Therefore, in this thesis, DoReFa-Net will be used as the method for weight quantization. DoReFa-Net presents a function of quantization that transforms the continuous weight values to discrete values ranging between 1 and -1, as demonstrated in Equation 1.11. This section will explore choosing the suitable quantization bit to achieve a balance between memory requirements and weight accuracy.

First, it is necessary to decide how many quantization bits to use for distinguishing between [1, -1]. Table 2.12 shows the accuracy of the proposed model when different quantization bits are used for each classifier. With 7-bit weight quantization, there is a

slight decrease in accuracy. However, when the weights are quantized to 6 bits or fewer, a more significant drop in accuracy occurs. Therefore, the proposed model will use 7-bit quantization for the weights.

Table 2.12 The accuracy of weight quantization with different bits in DoReFa-Net.

| Bits | Coarse accuracy | Fine accuracy |
|---|---|---|
| FP32 | 87.53 % | 90.21 % |
| 9 | 87.21 % | 89.95 % |
| 8 | 87.05 % | 89.87 % |
| **7** | **86.81 %** | **89.72 %** |
| 6 | 85.23 % | 88.48 % |
| 5 | 83.40 % | 86.28 % |
| 4 | 63.18 % | 70.36 % |

Next, to speed up model computation, a direct fixed-point conversion is used and stored in memory instead of using a lookup table for conversion. Since DoReFa-Net quantizes the weights between 1 and -1, the integer bits for the weights are set to 2. The next step is to determine the number of decimal bits for the weights. From Table 2.13, it can be seen that when the decimal bits are set to 6, there is no significant drop in accuracy. However, when the decimal bits are reduced to 5 or fewer, a noticeable decrease in accuracy occurs. Therefore, each weight is represented by 8 bits, including 2 integer bits and 6 decimal bits.

Table 2.13 The accuracy of DoReFa-Net with various decimal bits.

| Integer bits | Decimal bits | Coarse accuracy | Fine accuracy |
|---|---|---|---|
| 2 | 9 | 86.78 % | 89.68 % |

| | | | |
|---|---|---|---|
| 2 | 8 | 86.75 % | 89.67 % |
| 2 | 7 | 86.41 % | 89.51 % |
| **2** | **6** | **86.27 %** | **89.43 %** |
| 2 | 5 | 84.75 % | 88.39 % |

Following the implementation of the DoReFa-Net technique and retraining of the proposed branch CNN model, the weights undergo conversion from 32-bit floating-point numbers to 8-bit fixed-point numbers. This conversion accelerates the inference speed for the proposed CNN model and reduces hardware costs.

## 2.6 Activation Quantization Method

Quantizing weights is the primary focus of most research, as maintaining model accuracy poses a challenge during activation quantization. Despite this, activations stored in the feature map consume more storage space than weights, highlighting the need for activation quantization. This section compares different methods for quantizing activations, recognizing the significance of minimizing the number of bits for activations in hardware computations when employing the convolutional neural network.

Firstly, the utilization of DoReFa-Net for activation quantization is introduced, as discussed in Section 1.7. It is imperative to restrict the activation range between 0 and 1 using a bounded function, as shown in Equation 1.13. The scaling factor of 0.0625 is chosen to serve as the bounded function to confine values within the range between 0 and 1, as illustrated in Equation 2.4.

$$h(x) = clip(x \times 0.0625, 0, 1) \tag{2.4}$$

After applying the bounded function to scale the activations, Equation 1.11 is utilized to carry out activation quantization, resulting in k-bit fixed-point numbers ranging between 0 and 1. Table 2.14 illustrates the accuracy of activations at various bit-width. The table reveals that accuracy remains relatively consistent when the bit-width is set to 7 bits. However, a notable decline in accuracy is noticeable when reducing the bit-width to 6 bits.

Table 2.14 The accuracy of activation quantization with different bits in DoReFa-Net.

| Bits | Coarse accuracy | Fine accuracy |
|---|---|---|
| FP32 | 86.27 % | 89.43 % |

| | | |
|---|---|---|
| 9 | 86.22 % | 89.41 % |
| 8 | 86.13 % | 89.36 % |
| **7** | **86.06 %** | **89.28 %** |
| 6 | 85.29 % | 87.62 % |

Nevertheless, employing Equation 1.11 for activation quantization necessitates an additional multiplier and divider in the hardware implementation, leading to a considerable increase in computational workload during model inference.

Furthermore, as discussed in Section 1.7, the use of PACT for activation quantization was explored. Unlike ReLU, which has no upper limit on output values and thus requires a sufficient precision range to accurately represent, PACT dynamically adjusts the upper bound of each layer's activation during training through backpropagation, and each layer have different upper bound value called alpha. However, this also results in varying memory requirements for feature maps across different layers, making it challenging to reuse memory in subsequent hardware implementations.

Table 2.15 The alpha value in each layer with 100 epochs and 200 epochs.

| **Alpha value** | **100 epochs** | **200 epochs** |
|---|---|---|
| Layer 1 | 1.3719286 | 1.3830165 |
| Layer 2 | 1.3525951 | 1.3779949 |
| Layer 3 | 2.6114667 | 2.6246102 |
| Layer 4 | 1.1478527 | 1.1531808 |
| Layer 5 | 2.0920019 | 2.0938022 |
| **Coarse accuracy** | 86.21% | 86.24% |
| **Fine accuracy** | 89.27% | 89.36% |

From Table 2.15, it can be observed that although the alpha values of each layer vary slightly between 100 and 200 epochs, the model's accuracy only decreases by 0.03% to 0.09%. Additionally, the integer range of alpha values for each layer falls between 1 and 3. Therefore, the proposed method first utilizes PACT to train the model for a small number of epochs to determine the upper bounds of each layer, and then selects the maximum value as upper bound for each layer to retrain the model. The maximum alpha value for the proposed model is 3, so 3 is used as the upper bound for each layer during retraining. From Table 2.16, it can be seen that although the model's accuracy with a fixed alpha value of 3 is slightly lower than PACT by 0.07% to 0.1%, it reduces the complexity of subsequent hardware implementations.

Table 2.16 The accuracy of setting different alpha value.

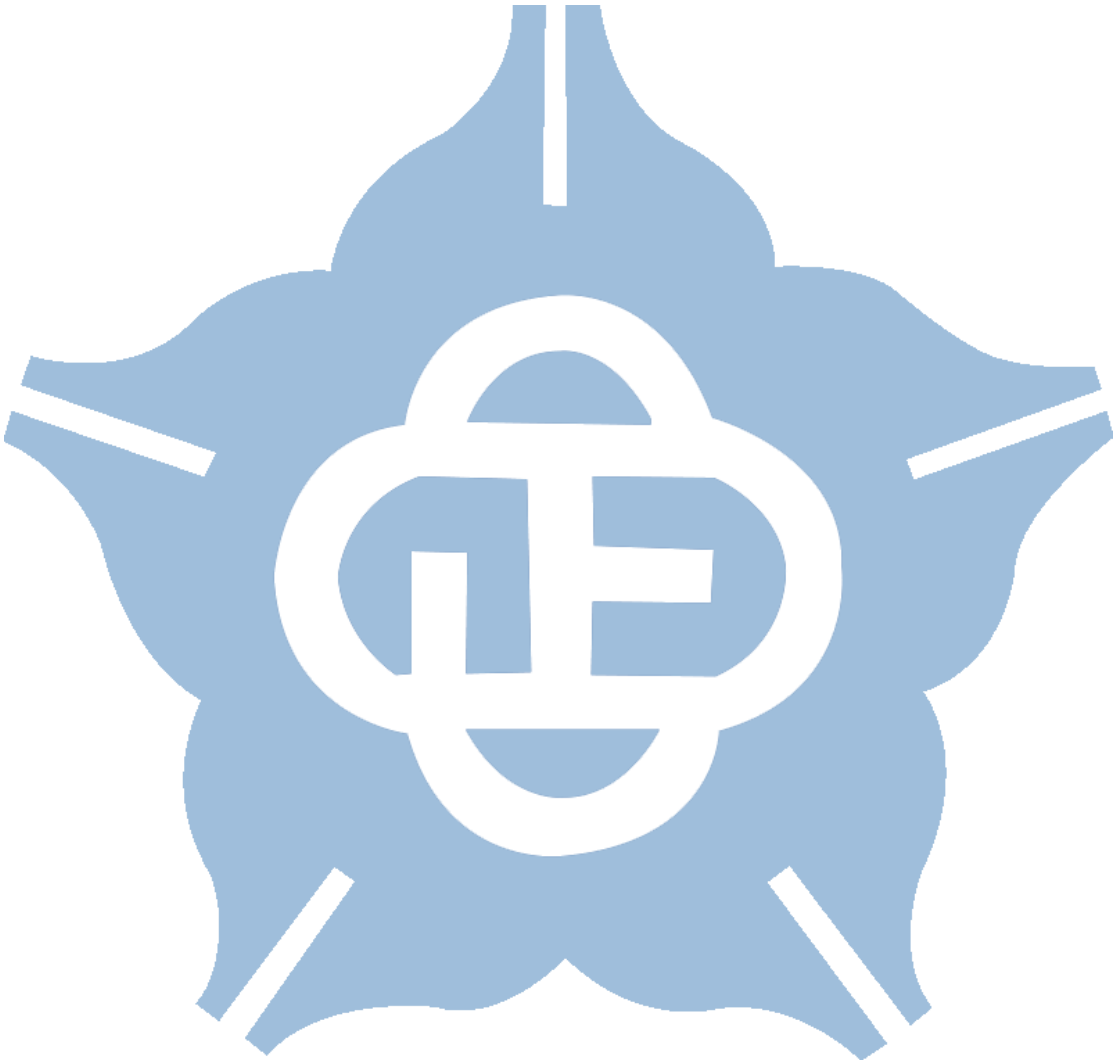
| | **Clipping value to 3** | **PACT** |
|---|---|---|
| **Coarse accuracy** | 86.14% | 86.21% |
| **Fine accuracy** | 89.17% | 89.27% |

Since the upper bound value of activation is set to 3 in each layer, so the integer part is represented as an unsigned number with 2 bits. From Table 2.17, the accuracy of activations is showcased at varying decimal precision. The table reveals that when the decimal precision is set to 6 bits, the accuracy remains relatively stable. However, a noticeable decline in accuracy is noted when the decimal precision is reduced to 5 bits. Hence, it is determined to maintain the decimal precision at 6 bits, enabling activation values to be stored with only a quarter of the memory required for original FP32 storage.

Table 2.17 The accuracy of activation quantization with different decimal bits.

| **Integer bits** | **Decimal bits** | **Coarse accuracy** | **Fine accuracy** |
|---|---|---|---|
| 2 | 9 | 86.14 % | 89.17 % |

| | | | |
|---|---|---|---|
| 2 | 8 | 86.05 % | 89.15 % |
| 2 | 7 | 85.85 % | 89.08 % |
| **2** | **6** | **85.57 %** | **89.02 %** |
| 2 | 5 | 84.26 % | 87.68 % |



## 2.7 Software Result

The confusion matrix is widely used in machine learning and pattern recognition to evaluate the performance of classification models. It displays the correspondence between the model's predictions and the actual results for different classes. By observing the confusion matrix, one can assess the model's performance on each class and understand its classification performance for different categories.

Table 2.18 depicts the confusion matrix of the coarse classifier. As mentioned in Section 2.4, due to the inclusion of an early exit mechanism for the Normal class, the accuracy of the coarse classifier for the Normal class must be sufficiently high. This enables the UAV to conserve energy through early exit during practical applications, thereby extending the flight time for disaster detection.

Table 2.18 The confusion matrix of coarse classifier in the proposed model.

| | | Predicted | | | Accuracy |
|---|---|---|---|---|---|
| | | **Disaster** | **Normal** | **Accident** | |
| **Actual** | **Disaster** | 178 | 23 | 8 | 85.16% |
| | **Normal** | 29 | 801 | 48 | 91.23% |
| | **Accident** | 12 | 25 | 162 | 81.41% |

Table 2.19 illustrates the confusion matrix of the fine classifier in the proposed model. It can be observed that the accuracy of the Normal class is the highest, as it constitutes the majority of the dataset. Traffic accidents exhibit less distinct features compared to Fire and Flood, resulting in lower accuracy.

Table 2.19 The confusion matrix of fine classifier in the proposed model.

| | | Predicted | | | | | Accuracy |
|---|---|---|---|---|---|---|---|
| | | **Collapsed building** | **Fire** | **Flood** | **Nomal** | **Traffic accidents** | |
| **Actual** | **Collaps** | 83 | 0 | 3 | 10 | 6 | 81.37% |
| | **Fire** | 0 | 92 | 5 | 6 | 1 | 88.46% |
| | **Flood** | 1 | 5 | 90 | 8 | 1 | 85.71% |
| | **Nomal** | 22 | 4 | 6 | 830 | 16 | 94.53% |
| | **Traffic** | 7 | 1 | 1 | 11 | 77 | 79.38% |

After repeated experiments and analysis, the hyperparameters of the proposed branch CNN model have been determined. As shown in Table 2.20.

Table 2.20 Hyperparameters setting ad loss function.

| **Hyperparameter** | **Value** |
|---|---|
| Batch Size | 32 |
| Train epoch | 200 |
| Retrain epoch | 200 |
| Learning rate | 0.001 |
| Optimization algorithm | SGD |
| Loss function | categorical_crossentropy |

# Chapter 3 Hardware Implementation

## 3.1 Fixed-point of Batch Normalization

After the quantization of weight and activation, the next step is to perform fixed-point quantization on the parameters of batch normalization. This helps to reduce the computational load and memory requirement for subsequent hardware implementation.

Once the model training is complete, each layer of batch normalization generates four parameters: mini-batch mean, mini-batch variance, γ, and β, as demonstrated in Equations 1.4, 1.5, 1.6, and 1.7. To reduce the complexity of hardware calculations later on, these four parameters are simplified into two using mathematical formulas, as shown in Equations 3.1 and 3.2, where $\mu_B$ is the mini-batch mean and $\alpha_B^2$ is the mini-batch variance. Therefore, the batch normalization calculations can be simply carried out using multiplication and addition, as illustrated in Equation 3.3.

$$\gamma' = \frac{1}{\sqrt{\alpha_B^2}}\gamma \tag{3.1}$$

$$\beta' = -(\mu_B\gamma') + \beta \tag{3.2}$$

$$y_i = \gamma' x_i + \beta' \tag{3.3}$$

Next, the simplified batch normalization parameters need to undergo fixed-point quantization. Starting with γ, since its value ranges from 0 to 1, 2 bits are used to represent the integer and sign bits. The next step is to determine the bit-wdith of decimal part. As observed in Table 3.1, accuracy does not significantly decrease when the decimal bits range from 11 to 9. However, when the decimal bits are 8 or lower, accuracy drops noticeably. Therefore, each value of γ can be represented using 11 bits, including 2 integer bits and 9 decimal bits.

Table 3.1 The test accuracy for various decimal bits of $\gamma'$.

| Integer bits | Decimal bits | Coarse accuracy | Fine accuracy |
|---|---|---|---|
| 2 | 11 | 85.48 % | 88.96 % |
| 2 | 10 | 85.43% | 88.82% |
| **2** | **9** | **85.36%** | **88.69%** |
| 2 | 8 | 82.75% | 87.12% |
| 2 | 7 | 77.57% | 82.43% |
| 2 | 6 | 70.81% | 76.53% |

After completing the fixed-point quantization for $\gamma'$, the similiar process continues with $\beta'$. Since $\beta'$ ranges from -3 to 3, a total of 3 bits are used to represent the integer and sign bits. The next step is to determine the bit-width for the decimal part of $\beta'$. From Table 3.2, it is observed that accuracy remains stable when the decimal bits range from 8 to 6. However, when the decimal bits are reduced to 5 or lower, accuracy significantly drops. Therefore, $\beta'$ is represented using 3 integer bits and 6 decimal bits for fixed-point quantization.

Table 3.2 The test accuracy for various decimal bits of $\beta'$.

| Integer bits | Decimal bits | Coarse accuracy | Fine accuracy |
|---|---|---|---|
| 3 | 8 | 85.34 % | 88.65 % |
| 3 | 7 | 85.25% | 88.54% |
| **3** | **6** | **85.12%** | **88.38%** |
| 3 | 5 | 83.75% | 87.43% |
| 3 | 4 | 78.69% | 82.43% |

Finally, Table 3.3 summarizes the number of bits required for each parameter for the following hardware implementation. Compared to traditional models that use 32-

bit floating points for computation, the proposed model uses fewer bits. This reduction helps lower power consumption and improve inference speed.

Table 3.3 The selection of fixed-point for hardware implementation.

| | **Integer bits** | **Decimal bits** | **Total bits** |
|---|---|---|---|
| **Weight** | 2 | 6 | 8 |
| **Activation** | 2 | 6 | 8 |
| **γ′ of batch normalization** | 2 | 9 | 11 |
| **β′ of batch normalization** | 3 | 6 | 9 |

# 3.2 B-CNN Hardware Accelerator Architecture

This section will provide a detailed explanation of the hardware implementation and memory usage. Figure 3.1 shows the proposed B-CNN block diagram with early exit mechanism.

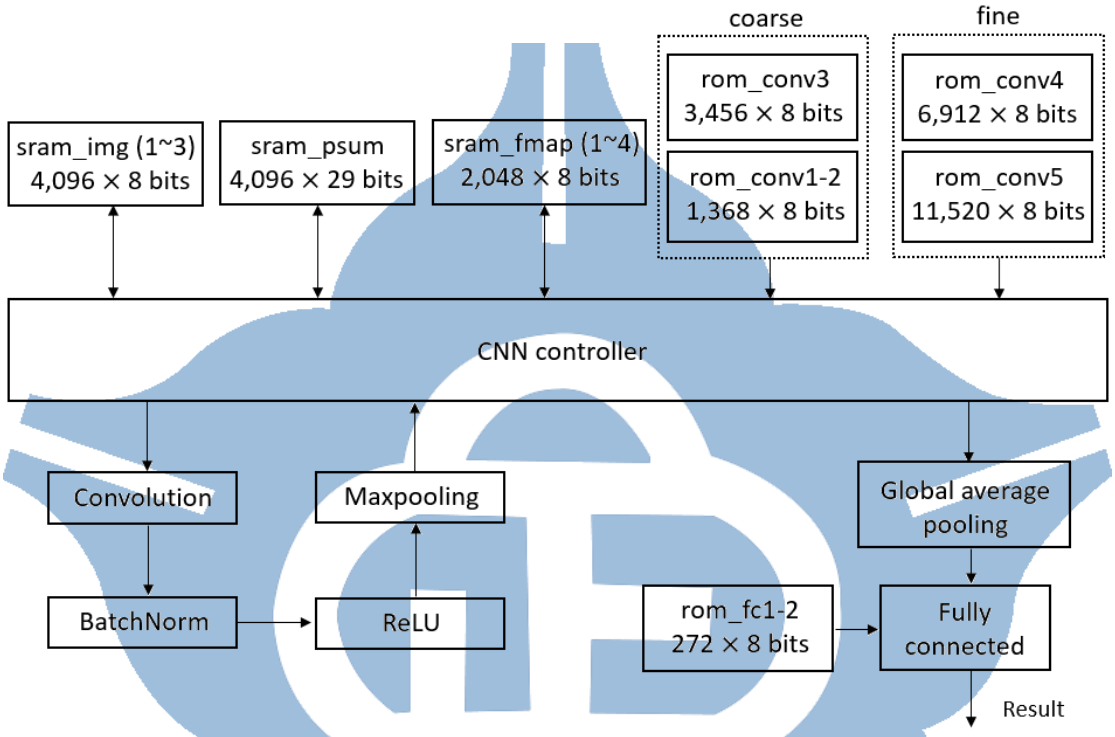



Figure 3.1 The block diagram of proposed B-CNN model.

The storage space needed for the entire hardware computation mainly consists of two types of memory: SRAM and ROM. SRAM is used to store the input image, the partial sums during computation, and the feature maps after each convolution layer. ROM is used to store the weights needed for the convolution and fully connected layers.

The convolution block is responsible for applying zero-padding to the input image and feature map, and performing convolution operations with the weights. The batch normalization block handles the multiplication and addition of the convolution results using two parameters, $\gamma'$ and $\beta'$, as shown in Equation 3.3. These parameters are stored using a look-up table, so additional ROM is not needed. The ReLU block sets

negative values to zero. The max pooling block performs the pooling process and reduces the size of the feature map. Finally, the global average pooling block calculates the average of the input channel and sends the result to the fully connected block for model prediction.

Table 3.4 The memory usage for each layer.

| **Stored data** | **Feature map size (width × height × channel × bit-width)** | **Total size (bits)** | **Data management** |
|---|---|---|---|
| Input image | 64 × 64 × 3 × 8 | 98,304 | sram_img1, sram_img2, sram_img3 |
| Feature maps of layer 1 | 32 × 32 × 8 × 8 | 65,536 | sram_fmap1, sram_fmap2, sram_fmap3, sram_fmap4 |
| Feature maps of layer 2 | 16 × 16 × 16 × 8 | 32,768 | sram_img1 |
| Feature maps of layer 3 | 8 × 8 × 24× 8 | 12,288 | sram_fmap1 |
| Feature maps of layer 4 | 4 × 4 × 32 × 8 | 4,096 | sram_fmap2 |
| Feature maps of layer 5 | 2 × 2 × 40 × 8 | 1,280 | sram_fmap1 |

Table 3.4 shows the memory usage for each layer. First, the input image is a 64×64 color image with R, G, and B channels, and each pixel value is represented by 8 bits. The image is stored in three SRAMs: sram_img1, sram_img2, and sram_img3, totaling 98,304 bits, which is the largest memory requirement in the entire computation process. After loading the input image into the SRAM, the convolution operations for each layer

begin. The feature map from the first layer is stored in four SRAMs: sram_fmap1, sram_fmap2, sram_fmap3, and sram_fmap4, totaling 65,536 bits, making it the largest feature map. The size of subsequent feature maps is halved by max-pooling, so from the third layer onward, the feature maps are mainly stored alternately in sram_fmap1 and sram_fmap2, without requiring additional storage space.

By reusing memory, the memory requirements during hardware computation are reduced. Unused memory will be managed with power gating to save energy. The details of the power gating technique are explained in Section 3.3.

The proposed B-CNN model is mainly divided into coarse and fine prediction stages. The number of cycles needed to complete the model predictions is shown in Figure 3.2. Performing model predictions using B-CNN requires a total of 2,350,000 cycles. However, the coarse prediction only needs 1,850,000 cycles. Therefore, for normal class data, an early exit can be used to reduce computation and power consumption. For other disaster or accident categories, deeper computations are necessary for higher accuracy, but after the coarse classifier, only an additional 500,000 cycles are needed to complete the process, as described in Section 2.4.

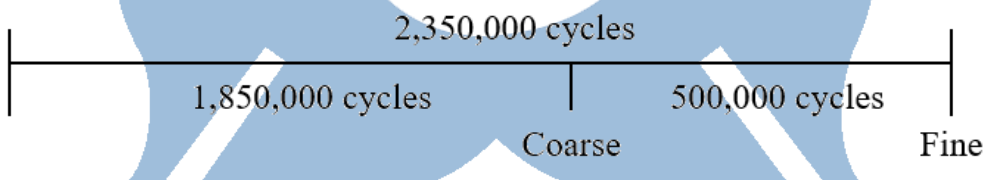



Figure 3.2 The different cycle counts for coarse and fine prediction in Verilog simulation phase.

## 3.3 Power Gating in Memory Management

Power gating is a crucial technique in modern memory management, aimed at enhancing energy efficiency in electronic systems. By selectively cutting off power to inactive parts of a circuit, it significantly reduces both dynamic and static power consumption. In memory systems, power gating achieves energy savings by disabling unused memory blocks. This ensures that only necessary memory sections are powered on, tailored to the current operational needs. Control circuits play a pivotal role in managing the power supply, determining when to activate or deactivate specific memory sections based on usage patterns. This technique not only boosts energy efficiency but also reduces heat generation, which is critical for maintaining device reliability and longevity.

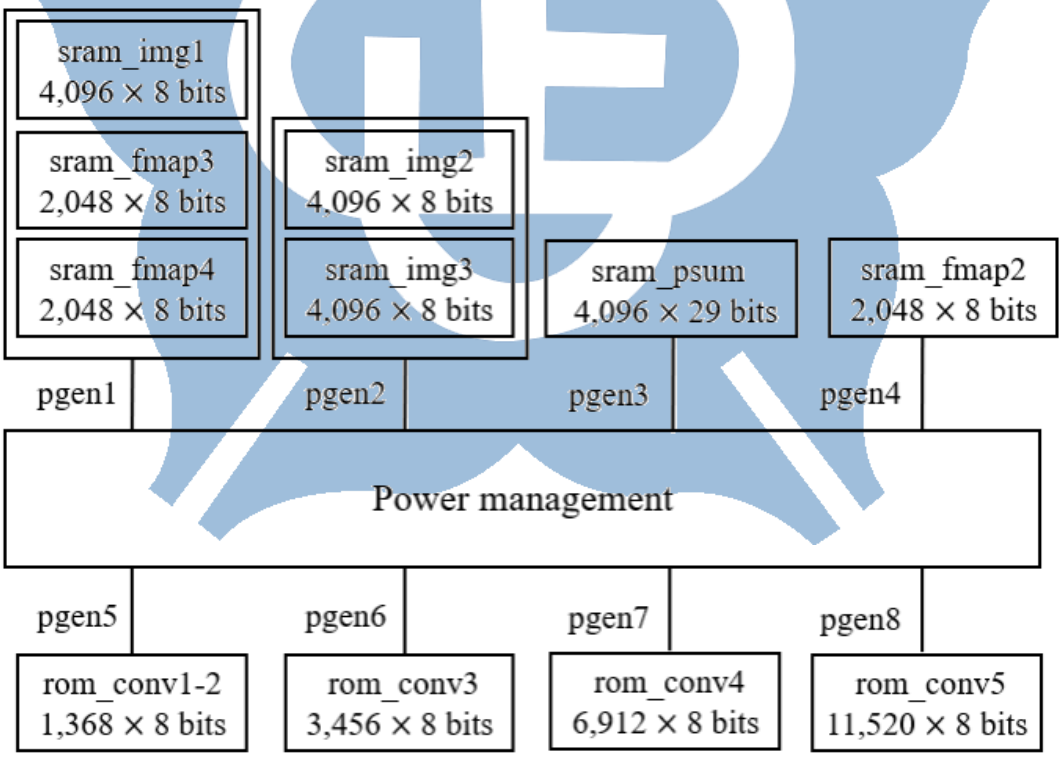



Figure 3.3 The power gating control for memory management.

The proposed memory power gating method is shown in Figure 3.3. The power control module, called power management, generates control signals pgen1 to pgen8 to manage the power switches for different memories. Due to the same circuit activity, sram_img1, sram_fmap3, and sram_fmap4 are controlled by the same signal, pgen1, while sram_img2 and sram_img3 are controlled by pgen2. Further details on the memory power management for the entire hardware architecture are provided in Table 3.5.

Table 3.5 The power state in each layer.

| **layer** | **Power domain** | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | **pgen1** | **pgen2** | **pgen3** | **pgen4** | **pgen5** | **pgen6** | **pgen7** | **pgen8** |
| **layer1** | On | On | On | On | On | Off | Off | Off |
| **layer2** | On | Off | On | On | On | Off | Off | Off |
| **layer3** | On | Off | On | Off | Off | On | Off | Off |
| **layer4** | Off | Off | On | On | Off | Off | On | Off |
| **layer5** | Off | Off | Off | On | Off | Off | Off | On |

The proposed B-CNN hardware computation flow is shown in Figure 3.4. The process is separated into a three-layer coarse prediction and a five-layer fine prediction after loading the input image. Each layer follows a similar computation process, including convolution, batch normalization, ReLU, max-pooling, and memory read/write operations. The final prediction result, whether coarse or fine, is obtained by first applying global average pooling to the feature map, followed by a fully connected layer.

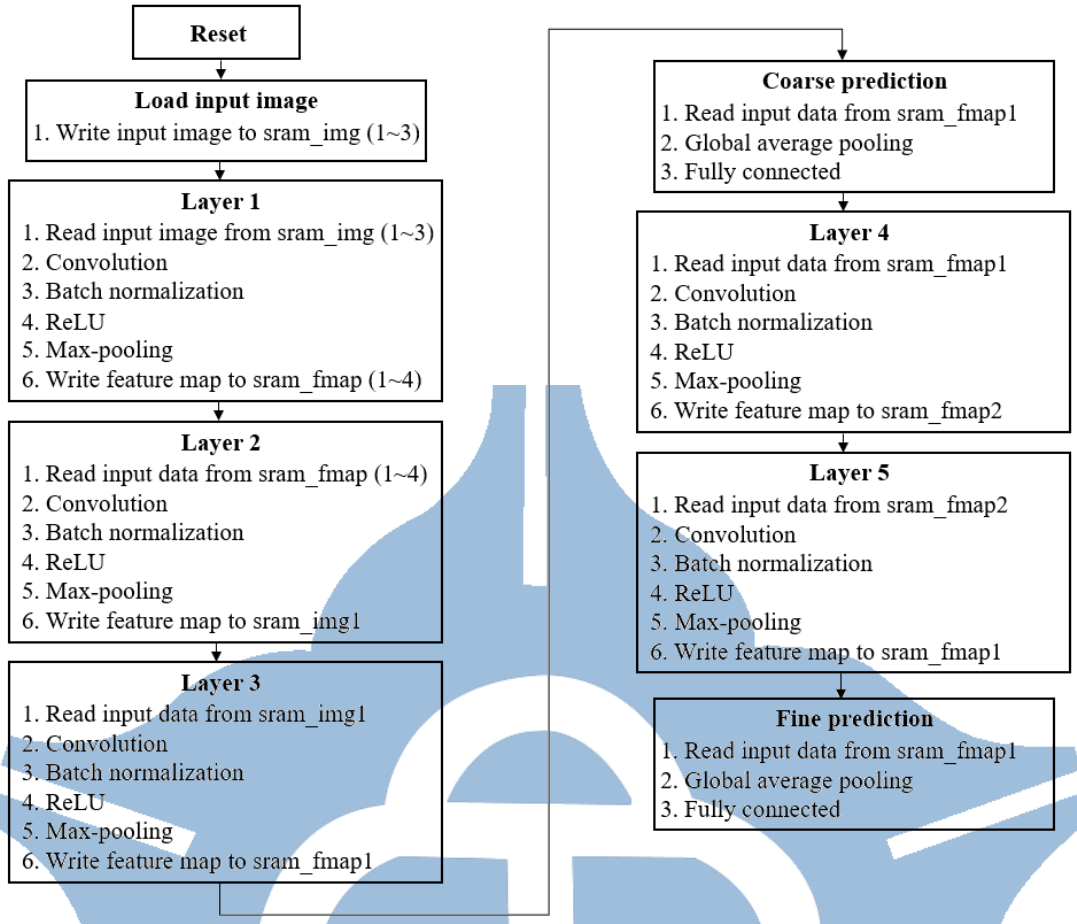



Figure 3.4 Flow chart for the hardware implementation of proposed B-CNN design.

## 3.4 Analysis of B-CNN Hardware Implementation

In this section, an analysis of the experimental results for the proposed B-CNN hardware design will be conducted. From Figure 3.5, the proposed B-CNN hardware design with power-gating consumes 37.56 mW at 500 MHz. This results in about a 12.9% reduction in power consumption compared to the design without the power-gating technique. Therefore, the experimental results demonstrate the effectiveness of power gating in achieving low power consumption for the circuit.

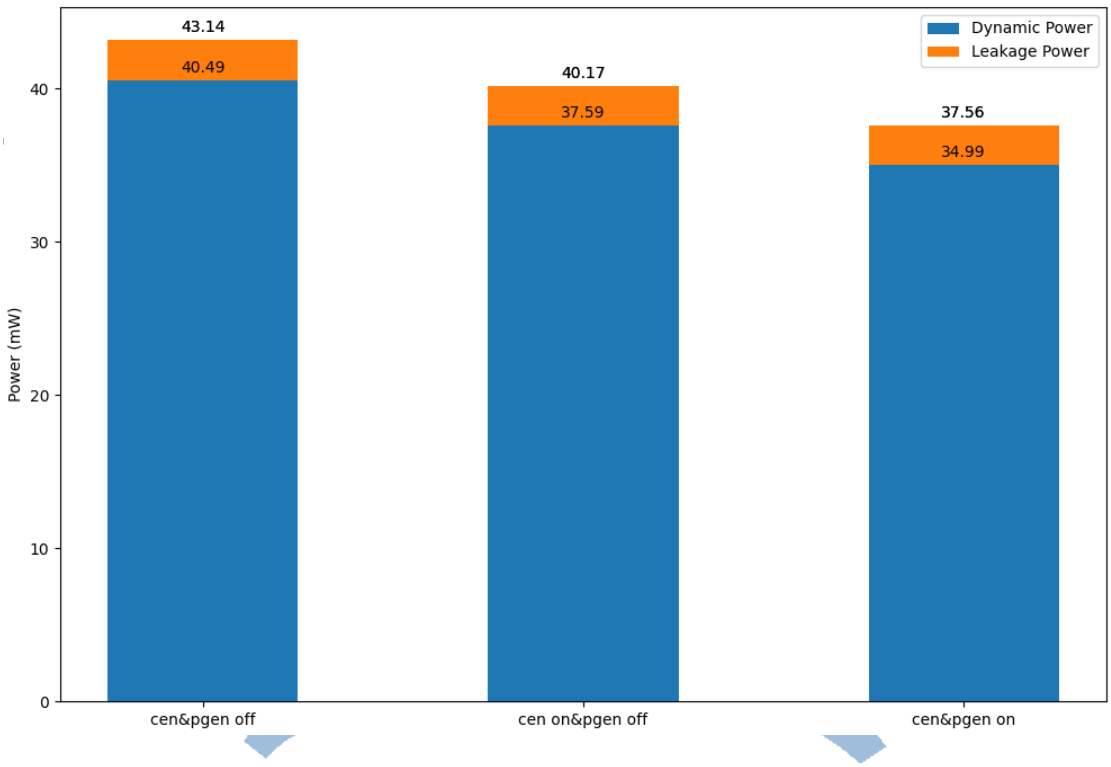



Figure 3.5 The power management for power gating control.

Figure 3.6 provides a detailed view of the impact of the power gating technique on each memory block. It shows that both SRAM and ROM reduce power consumption after applying power gating. Therefore, adding power gating is beneficial for memory power management.

```
------------------------------------------------------------
    Leakage    Internal   Switching       Total Instance
------------------------------------------------------------
1.40284e-04 4.30162e-04 3.60324e-07 5.70807e-04 /CNN/u_sram_img_1
1.40284e-04 1.41328e-04 1.18173e-07 2.81730e-04 /CNN/u_sram_img_2
1.40284e-04 1.41397e-04 1.41196e-07 2.81822e-04 /CNN/u_sram_img_3
2.55149e-04 1.13103e-04 2.43686e-08 3.68277e-04 /CNN/u_sram_partial_sum
1.04925e-04 2.50457e-04 3.56254e-07 3.55738e-04 /CNN/u_sram_feature_map_1
1.04925e-04 1.72652e-04 1.91251e-07 2.77768e-04 /CNN/u_sram_feature_map_2
1.04925e-04 1.06583e-04 1.08970e-07 2.11617e-04 /CNN/u_sram_feature_map_3
1.04925e-04 1.06617e-04 1.19939e-07 2.11662e-04 /CNN/u_sram_feature_map_4
6.10858e-05 4.45750e-06 6.76170e-10 6.55440e-05 /CNN/u_rom_conv12
7.68290e-05 4.65629e-06 6.49509e-10 8.14859e-05 /CNN/u_rom_conv3
1.03396e-04 5.01957e-06 1.31027e-09 1.08417e-04 /CNN/u_rom_conv4
1.38818e-04 5.62741e-06 9.65435e-09 1.44455e-04 /CNN/u_rom_conv5
------------------------------------------------------------
1.47583e-03 1.48206e-03 1.43277e-06 2.95932e-03 Total
------------------------------------------------------------
```

(a)

```
------------------------------------------------------------
    Leakage    Internal   Switching       Total Instance
------------------------------------------------------------
1.22589e-04 3.40908e-04 9.50287e-08 4.63593e-04 /CNN/u_sram_img_1
6.87327e-05 1.96644e-05 1.16619e-07 8.85137e-05 /CNN/u_sram_img_2
6.87327e-05 1.96741e-05 1.39337e-07 8.85461e-05 /CNN/u_sram_img_3
2.34028e-04 8.95593e-05 2.27560e-08 3.23610e-04 /CNN/u_sram_partial_sum
1.04925e-04 2.51985e-04 3.70074e-07 3.57280e-04 /CNN/u_sram_feature_map_1
8.84426e-05 1.20874e-04 1.88004e-07 2.09505e-04 /CNN/u_sram_feature_map_2
7.58706e-05 5.45435e-05 1.09731e-07 1.30524e-04 /CNN/u_sram_feature_map_3
7.58706e-05 5.45610e-05 1.19811e-07 1.30551e-04 /CNN/u_sram_feature_map_4
6.10858e-05 4.45763e-06 3.87580e-09 6.55473e-05 /CNN/u_rom_conv12
6.19186e-05 3.67111e-06 6.54203e-10 6.55904e-05 /CNN/u_rom_conv3
9.71699e-05 4.69441e-06 1.31963e-09 1.01866e-04 /CNN/u_rom_conv4
1.38818e-04 5.49919e-06 8.82891e-09 1.44326e-04 /CNN/u_rom_conv5
------------------------------------------------------------
1.19818e-03 9.70092e-04 1.17604e-06 2.16945e-03 Total
------------------------------------------------------------
```

(b)

Figure 3.6 The power consumption of each memory block (a) Without power-gating (b) With power gating.

Table 3.6 illustrates the accuracy at various stages from software implementation to hardware implementation. It can be observed that the B-CNN model, after quantization and fixed-point representation, reduces the numerical range that each parameter can represent, resulting in a gradual decrease in accuracy by approximately 2.1%. During the Verilog RTL simulation phase of the proposed B-CNN hardware,

some bit errors during computations led to a decrease in accuracy. However, the overall accuracy still remains above 88%.

Table 3.6 The test accuracy with different operations.

| **Operation of each stage** | **Coarse accuracy** | **Fine accuracy** |
|---|---|---|
| Build B-CNN model | 87.53% | 90.21 % |
| Weight quantization to 8 bits | 86.27% | 89.43% |
| Activation quantization to 8 bits | 85.57% | 89.02% |
| Convert $\gamma'$ in BN to 11 bits | 85.24% | 88.63% |
| Convert $\beta'$ in BN to 9 bits | 85.02% | 88.38% |
| Verilog Register Transfer Level | 84.85% | 88.18 % |

Table 3.7 presents the test accuracy and classification results during the Verilog RTL simulation stage. After model quantization and fixed-point conversion, the best classification results are for the Normal category, with accuracy remaining above 90%, reaching 93.17%. The worst classification results are for the Traffic accident category, with an accuracy of only 73.20%, showing a similar prediction trend to the software stage.

Table 3.7 The test accuracy and classification results in Verilog RTL simulation.

| **Image type** | **Image number (correct / total)** | **Test accuracy** |
|---|---|---|
| Collapsed building | 75 / 102 | 73.52 % |
| Fire | 84 / 104 | 80.77% |
| Flood | 86 / 105 | 81.90% |
| Normal | 818 / 878 | 93.17% |

| | | |
|---|---|---|
| Traffic accident | 71 / 97 | 73.20% |
| **Overall accuracy** | 1,134 / 1,286 | 88.18% |

After multiple adjustments and testing, the proposed B-CNN hardware circuit achieves a maximum operating frequency of 500 MHz. As shown in Equations 3.3, the model prediction time requires a total of 0.0047 seconds. Equations 3.4 and 3.5 provide the calculation times required for each classifier, with coarse classifier taking 0.0037 seconds and fine classifier taking 0.001 seconds. Therefore, UAVs can perform disaster detection tasks more quickly and efficiently, conserving power in the process.

$$2{,}350{,}000 \times 2\ \text{ns} = 4{,}700{,}000\ \text{ns} \approx 0.0047\ \text{s} \tag{3.3}$$

$$1{,}850{,}000 \times 2\ \text{ns} = 3{,}700{,}000\ \text{ns} \approx 0.0037\ \text{s} \tag{3.4}$$

$$500{,}000 \times 2\ \text{ns} = 1{,}000{,}000\ \text{ns} \approx 0.001\ \text{s} \tag{3.5}$$

The chip layout of the proposed B-CNN hardware design is implemented using TSMC 16nm CMOS technology. As shown in Figure 3.7, it indicates the positions of ROM, SRAM, and PSUM. The chip area is $0.75 \times 0.75\ mm^2$.

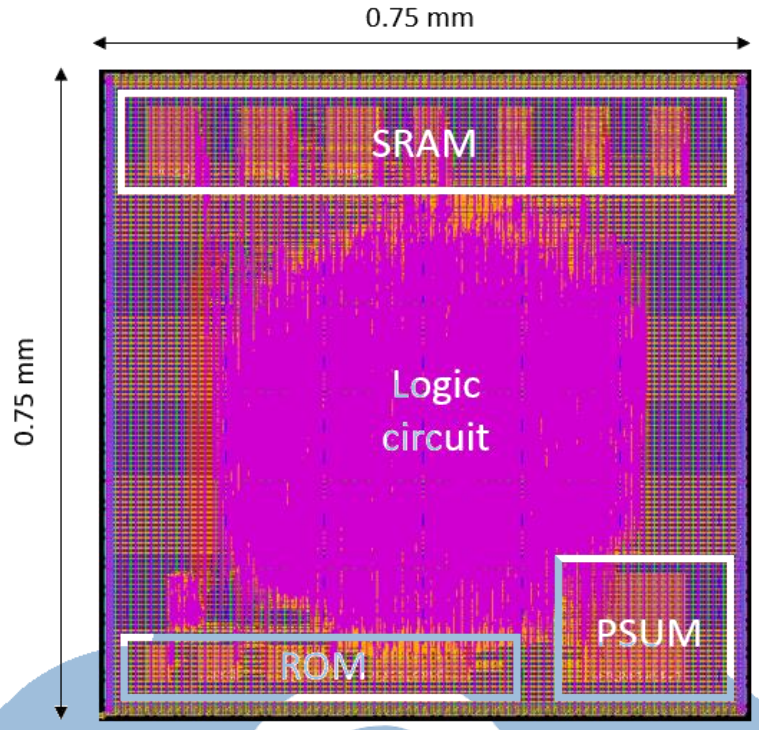



Figure 3.7 The layout of the proposed B-CNN hardware design.

The implementation results of the proposed B-CNN hardware design are shown in Table 3.8. The maximum operating frequency is 500 MHz, the power comsumption is 37.56 mW, and the memory size is 57 KB. The proposed B-CNN hardware design is implemented in TSMC 16nm CMOS technology.

Table 3.8 The B-CNN hardware implementation results.

| | **The proposed architecture** |
|---|---|
| **Technology** | TSMC 16nm CMOS |
| **Frequency (MHz)** | 500 |
| **Power (mW)** | 37.56 |
| **Chip area ($mm^2$)** | 0.5625 |
| **Parameters** | 24,008 |
| **Memory size (KB)** | 57 |

Additionally, before the circuit can perform convolution operations, it must first read the weight values from ROM, and this operation must be completed within half a cycle. However, reading the weight values from ROM requires waiting for a certain access time. As observed in Table 3.9, since rom_conv5 has the highest number of bits, it requires the longest access time, totaling 0.8429 ns, which is also the critical path of the proposed B-CNN hardware.

Table 3.9 The access time of ROM.

| **Memory name** | **Memory size (bits)** | **Access time (ns)** |
|---|---|---|
| rom_conv12 | 10,944 | 0.6482 |
| rom_conv3 | 27,648 | 0.7005 |
| rom_conv4 | 55,296 | 0.7611 |
| rom_conv5 | 92,160 | 0.8429 |

## 3.5 Summary

Table 3.10 provides a detailed comparison of different methods used for processing the AIDER dataset, including various hardware information, image sizes, quantization methods, inference times, memory sizes, FLOPs, and accuracies.

Table 3.10 Comparison with prior methods.

| | **[21] IEEE JSTARS'2020** | **[7] IEEE CVPRW'2019** | **[23] IEEE LGRS'2023** | **[22] IEEE EUVIP'2022** | **Proposed work** |
|---|---|---|---|---|---|
| **Dataset** | AIDER | AIDER | AIDER | AIDER | AIDER |
| **Architecture** | EmergencyNet | ERNet | WATT-EffNet | CNN | Branch CNN |
| **Hardware information** | ARM Cortex-A57 | Intel i7 | Nvidia Tesla T4 GPU + TPU | Intel i5 | 16nm ASIC |
| **Image size** | 240 × 240 | 240 × 240 | 224 × 224 | 100 × 100 | 64 × 64 |
| **Quantization method** | No | No | No | No | DoReFa-Net + Fixed-point |
| **Inference time (s)** | 0.041 | 0.018 | N/A | 0.069 | 0.0037 (coarse) / 0.0047 (fine) |
| **Memory size (KB)** | 360 | 300 | 2,690 | 3,072 | 57 |
| **FLOPs** | 57M | N/A | N/A | 22M | 6.1M (coarse) / 7.4M (fine) |
| **Accuracy** | 83.1% | 90.1% | 88.5% | 88% | 90.21% (software) / 88.18% (hardware) |

Since the power consumption details in [21, 7, 23, 22] are not provided, the power consumption figures mainly come from manufacturer-provided information. These

include 91 W for the Intel i7, 28 W for the Intel i5, and 70 W for the Nvidia T4 GPU. Additionally, the memory sizes are not mentioned in [23], which only gives the number of parameters required by the model. Therefore, the data in the comparison table assumes each parameter is represented as a 32-bit floating-point value and converts it into MB. Moreover, the accuracy of 83.1% for [21] was derived from the analysis in [23].

The proposed B-CNN achieves better performance in both inference time and memory size compared to [21, 7, 23, 22] by resizing images to 64×64 and employing model quantization techniques to reduce the number of parameters and the bits required for each parameter. Consequently, hardware implementation allows faster model predictions with less storage space. Although slightly lower in accuracy compared to [21, 7, 23] due to fewer parameters, the use of branch training methods enhances feature learning, enabling the overall performance to remain comparable.

# Chapter 4 Conclusion and Future works

## 4.1 Conclusion

In this thesis, a B-CNN model with an early exit mechanism is proposed for UAV disaster detection. A label tree is created for frequently confused classes in the dataset, allowing the model to enhance feature learning through branch training. This enables training and learning with fewer parameters without significantly losing accuracy. DoReFa-Net and a clipped function are used to limit the range of weights and activations, with fixed-point quantization determining the required decimal bits for each parameter.

In the software implementation, experimental results show that the proposed B-CNN model achieves 90.21% accuracy with just 24K parameters. Even after model quantization, the accuracy remains at 88.38%, using only 8 bits for weights and activations. Compared to traditional CNN models that use 32-bit floating point representation, this approach reduces storage space and computational complexity by 75%. Before the hardware implementation, the parameters and computations required for batch normalization are simplified using mathematical formulas. This allows batch normalization to be computed with two parameters and simple multiplication and addition, with fixed-point quantization applied to the parameters.

In the hardware implementation, power gating manages memory usage by shutting off the power to idle memory, reducing unnecessary power consumption and improving memory efficiency. The proposed B-CNN hardware design is implemented using TSMC 16 nm CMOS technology, operating at a frequency of 500 MHz with a power consumption of 37.56 mW and a classification accuracy of 88.18%.

## 4.2 Future Works

The AIDER dataset currently includes only four disaster and accident categories. In the future, adding more diverse disaster types, such as landslides and hurricanes, can enhance the dataset's variety. This will enable the model to learn and adapt to different disaster environments. Additionally, categorizing the severity of each disaster type in the dataset into mild, moderate, and severe can improve the model's functionality. Instead of merely identifying the disaster type, the model could also provide the severity level to rescue teams, allowing them to allocate resources and manage disaster more effectively.

In addition, despite employing DoReFa-Net for weight quantization in this thesis to reduce the number of bits, multiplication operations are still required to be executed by multipliers. Therefore, shift-based quantization offers a promising way to improve the efficiency of hardware implementations for deep learning models. By converting multiplication operations into bit-shift operations, this method can greatly reduce computational load and power usage. Future research should aim to optimize precision, develop adaptive schemes, integrate hardware-aware training, implement on various hardware platforms, and ensure robustness and generalization.

Furthermore, Dynamic Voltage and Frequency Scaling (DVFS) is a technique employed in electronic systems to dynamically adjust the voltage and clock frequency of a processor or other components based on the current workload or environmental conditions. Therefore, by integrating power gating for memory power management and combining it with DVFS technology, the proposed hardware design of B-CNN can operate with lower power consumption in various working scenarios, effectively enhancing hardware efficiency.

# Reference


[1] Hung Pham, Dalil Ichalal, and Said Mammar, “Complete coverage path planning for pests-ridden in precision agriculture using UAV,” *in Proceedings of IEEE International Conference on Networking, Sensing and Control (ICNSC)*, pp. 1-6, Oct. 2020.

[2] Shubo Wang, Yu Han, Jian Chen, Zichao Zhang, Guangqi Wang, and Nannan Du, “A deep-learning-based sea search and rescue algorithm by UAV remote sensing,” *in Proceedings of IEEE CSAA Guidance, Navigation and Control Conference (CGNCC)*, pp. 1-5, Aug. 2018.

[3] Muhammad-Haziq Mohd-Tajudin, Shahbe Mat-Desa, and Junaidi Abdullah, “Traffic density identification from UAV images using deep learning: A preliminary study,” *in Proceedings of International Conference on Software, Knowledge, Information Management and Applications (SKIMA)*, pp. 93-98, Dec. 2023.

[4] A V. Shubhasree, Praveen Sankaran, and C V Raghu, “UAV image analysis of flooded area using convolutional neural networks,” *in Proceedings of International Conference on Connected Systems & Intelligence (CSI)*, pp. 1-7, Aug. 2022.

[5] Hoai Nam Vu, Huong Mai Nguyen, Cuong Duc Pham, Anh Dat Tran, Khanh Nguyen Trong, Cuong Pham, and Viet Hung Nguyen, “Landslide detection with unmanned aerial vehicles,” *in Proceedings of International Conference on Multimedia Analysis and Pattern Recognition (MAPR)*, pp. 1-7, Oct. 2021.

[6] Alireza Shamsoshoara, Fatemeh Afghah, Abolfazl Razi, Liming Zheng, Peter Z Ful´e, and Erik Blasch, “Aerial imagery pile burn detection using deep learning: the FLAME dataset,” *Computer Networks*, vol. 193, Jul. 2021.

[7] DJI drones (https://store.dji.com/tw/content/long-range-drone).

[8] Nvidia JetsonNano (https://dlcdnets.asus.com/pub/ASUS/mb/AIOT/jetson-nano-devkit-datasheet.pdf).

[9] Christos Kyrkou, and Theocharis Theocharides, "Deep-learning-based aerial image classification for emergency response applications using unmanned aerial vehicles, arXiv:1906.08716 [cs.CV]," *arXiv.org*, Jun. 2019.

[10] What is a Convolutional Neural Network? (https://skyengine.ai/se/skyengine-blog/125-what-is-a-convolutional-neural-network).

[11] Convolutional Neural Networks: Diving into the theory of CNNs (https://medium.com/@ysfmtr13/convolutional-neural-networks-diving-into-the-theory-of-cnns-c3935aa4ffc6).

[12] Sergey Ioffe and Christian Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *in Proceedings of International Conference on Machine Learning (ICML)*, pp. 448-456, Jul. 2015.

[13] Min Lin, Qiang Chen, and Shuicheng Yan, "Network in network, arXiv:1312.4400v3 [cs.NE]," *arXiv.org*, Mar. 2014.

[14] Yann Lecun, Leon Bottou, Yoshua Bengio, and Patrick Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.

[15] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton, "ImageNet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84-90, May 2017.

[16] Karen Simonyan and Andrew Zisserman, "Very deep convolutional networks for large-scale image recognition, arXiv:1409.1556v6 [cs.CV]," *arXiv.org*, Apr. 2015.

[17] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich, "Going

deeper with convolutions," *in Proceedings of IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2015.

[18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, "Deep residual learning for image recognition," *in Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770-778, Jun. 2016.

[19] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications, arXiv:1704.04861v1 [cs.CV]," *arXiv.org*, Apr. 2017.

[20] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," *in Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6848-6856, Jun. 2018.

[21] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size, arXiv:1602.07360v4 [cs.CV]," *arXiv.org*, Nov. 2016.

[22] Christos Kyrkou, and Theocharis Theocharides, "EmergencyNet: efficient aerial image classification for drone-based emergency monitoring using atrous convolutional feature fusion," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 13, pp. 1687-1699, Mar. 2020.

[23] Muhammad Munsif, Hina Afridi, Mohib Ullah, Sultan Daud Khan, Faouzi Alaya Cheikh, and Muhammad Sajjad, "A lightweight convolution neural network for automatic disasters recognition," *in Proceedings of European Workshop on Visual Information Processing (EUVIP)*, pp. 1-6, Oct. 2022.

[24] Gao Yu Lee, Tanmoy Dam, Md Meftahul Ferdaus, Daniel Puiu Poenar, and Vu N.

Duong, “WATT-EffNet: a lightweight and accurate model for classifying aerial disaster images,” *IEEE Geoscience and Remote Sensing Letters*, vol. 20, pp. 1-5, Apr. 2023.

[25] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon, “CBAM: convolutional block attention module, arXiv:1807.06521 [cs.CV],” *arXiv.org*, Jul. 2018.

[26] Abhinav Goel, Sara Aghajanzadeh, Caleb Tung and Shuo-Han Chen, “Modular neural networks for low-power image Classification on embedded devices,” *ACM Transactions on Design Automation of Electronic Systems*, vol 26, no.1, pp. 1-35, Jan. 2021.

[27] Zhicheng Yan, Hao Zhang, Robinson Piramuthu, Vignesh Jagadeesh, Dennis DeCoste, Wei Di, and Yizhou Yu, “HD-CNN: hierarchical deep convolutional neural networks for large scale visual recognition,” *in Proceedings of IEEE International Conference on Computer Vision (ICCV)*, pp. 2740-2748, Feb. 2016.

[28] Xinqi Zhu, and Michael Bain, “B-CNN: branch convolutional neural network for hierarchical classification, arXiv:1709.09890 [cs.CV],” *arXiv.org*, Oct. 2017.

[29] Abhinav Goel, Caleb Tung, Xiao Hu, Haobo Wang, James C. Davis, George K. Thiruvathukal, and Yung-Hsiang Lu, “Low-power multi-camera object re-identification using hierarchical neural networks,” *in Proceedings of IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, pp. 1-6, Aug. 2021.

[30] Priyadarshini Panda, Abhronil Sengupta, and Kaushik Roy, “Conditional deep learning for energy-efficient and enhanced pattern recognition, arXiv:1509.08971 [cs.CV],” *arXiv.org*, Jan. 2016.

[31] Surat Teerapittayanon, Bradley McDanel, and H.T. Kung, “BranchyNet: fast inference via early exiting from deep neural networks,” *in Proceedings of*

*International Conference on Pattern Recognition (ICPR)*, pp. 2464-2469, Dec. 2016.

[32] Yu Cheng, Duo Wang, Pan Zhou, and Tao Zhang, "A survey of model compression and acceleration for deep neural networks, arXiv:1710.09282v9 [cs.LG]," *arXiv.org*, Jun. 2020.

[33] Song Han, Huizi Mao, and William J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding, arXiv:1510.00149v5 [cs.CV]," *arXiv.org*, Feb. 2016.

[34] Jimmy Ba, Rich Caruana, "Do deep nets really need to be deep?," *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, Dec. 2014.

[35] Tara N. Sainath, Brian Kingsbury, Vikas Sindhwani, Ebru Arisoy, and Bhuvana Ramabhadran, "Low-rank matrix factorization for deep neural network training with high-dimensional output targets," *in Proceedings of IEEE International Conference on Acoustics, Speech and Signal Processing*, pp. 6655-6659, Oct. 2013.

[36] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1, arXiv:1602.02830v3 [cs.LG]," *arXiv.org*, Mar. 2016.

[37] Fengfu Li, Bin Liu, Xiaoxing Wang, Bo Zhang, and Junchi Yan, "Ternary weight networks, arXiv:1605.04711v3 [cs.CV]," *arXiv.org*, Nov. 2022.

[38] Dongqing Zhang, Jiaolong Yang, Dongqiangzi Ye, and Gang Hua, "LQ-Nets: learned quantization for highly accurate and compact deep neural networks, arXiv:1807.10029 [cs.CV]," *arXiv.org*, Jul. 2018.

[39] Asit Mishra, Eriko Nurvitadhi, Jeffrey J Cook, and Debbie Marr, "WRPN: wide reduced-precision networks, arXiv:1709.01134 [cs.CV]," *arXiv.org*, Sep. 2017.

[40] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang,

Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan, “PACT: Parameterized clipping activation for quantized neural networks, arXiv:1805.06085v2 [cs.CV],” *arXiv.org*, Jul. 2018.

[41] Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, and Yuheng Zou, “DoReFa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients, arXiv:1606.06160v3 [cs.NE],” *arXiv.org*, Feb. 2018.